The Definitive Reference: Event Listener & Keyboard Key Detection Performance in Frontend JavaScript

Audience: authors of keyboard-shortcut libraries, editors, and frontend event-infrastructure code. **Scope:** DOM event-listener and `KeyboardEvent` semantics, browser engine internals (Blink/WebKit/Gecko), measured dispatch and matching costs, the published latency evidence, and the implementation practice of shipping shortcut engines. **Companion sandbox:** every measured number is reproducible from the benchmark harness at `/mnt/agents/output/key-bench/`. **Measurement platform:** Chromium 150.0.7871.181 (real Blink + V8, headless), driven by Node v20.20.2 via puppeteer-core. **Date:** 2026-08-14. **Companion volume:** a parallel reference covers DOM-querying performance (`querySelector` & friends); see the companion DOM-querying volume.

1. The Mental Model: What a Keystroke Costs

A keystroke is not one event and not one cost: it is a pipeline of engine work that terminates — optionally — in your JavaScript. This chapter fixes the vocabulary: what registration buys you, what the dispatch algorithm does before user code runs, what a `KeyboardEvent` can and cannot tell you, and where focus and IME redirect the flow. Chapters 2 and 3 descend into engine internals and measurements; everything here is semantics — the contracts that make those measurements interpretable.

1.1 Listener registration surface

1.1.1 `addEventListener` options, dedup, removal matching; `on*` handlers; `handleEvent` objects

Registration is per-(target, type): each `EventTarget` owns an event listener list, and that list is only scanned when an event of the matching type dispatches through the node — an empty list costs nothing at dispatch.[1] The full option surface, with the semantics that matter for cost accounting:

Option / mechanism	Semantics	Cost-relevant fact
capture (default false)	When true, the listener fires before any EventTarget beneath it in the tree;[2] at AT_TARGET, capture and bubble listeners both run.[1]	Capture listeners on window/document run *first* for every matching event anywhere in the tree — the standard hot path for global shortcut managers.
once (default false)	"The listener will only be invoked once after which the event listener will be removed."[1]	Auto-removal happens at invoke time; add-time dedup still applies.
passive (default false,	The listener "will never call	The scroll-unblocking contract:

Option / mechanism	Semantics	Cost-relevant fact
with exceptions)	preventDefault()"; if it does, the canceled flag is not set and a console warning may be generated.[2] [3]	with passive: true the compositor may scroll without waiting for the main thread. Never applies to keyboard (§1.2.2).
signal (AbortSignal)	"The event listener will be removed when signal is aborted."[1]	O(1)-ish teardown of an entire shortcut scope (e.g., a modal) via one abort(), without storing each callback reference.
on* IDL attribute	Acts as a *non-capture* event listener;[4] exactly one handler per (object, type) slot — assignment overwrites, null clears.	No capture/passive/once/signal control; libraries must use addEventListener to coexist with other owners.[2]
handleEvent object	Any object with a handleEvent() method is a valid listener; this inside it is the object itself.[2] [5]	No closure or .bind() allocation per registration; identity-based removal is robust. Works only with addEventListener, not on* properties.[5]

Two matching rules govern the list, and both are classic traps. Dedup: a listener "is **not appended if it has the same type, callback, and capture**"[1] — `passive`, `once`, and `signal` are *not* part of the key, so re-adding with different values silently updates nothing. Symmetrically, `removeEventListener` matches on (type, callback, capture) only — "the only option `removeEventListener()` checks is the `capture`/`useCapture` flag."[6] The practical consequence:

```
// TRAP: anonymous callbacks defeat both dedup and removal.
el.addEventListener('keydown', e => onKey(e));   // registered
el.addEventListener('keydown', e => onKey(e));   // registered AGAIN
(no dedup)
el.removeEventListener('keydown', e => onKey(e)); // removes NOTHING
// FIX: keep a stable reference, or pass { signal } from an
AbortController.
```

Invocation order across mechanisms is registration order of the *listener entries*: interleaving `addEventListener('click', a)`, `onclick = b`, `addEventListener('click', c)` fires a, b, c, because setting the IDL attribute moves or creates a single entry.[4]

1.1.2 The five cost layers of a keystroke

We frame the cost of one keystroke as five sequential layers; every claim in this book's measurements (Ch. 3) maps onto one of them:

1. **OS/input pipeline → renderer.** Key scanning, layout resolution, IME arbitration, cross-process delivery into the main thread. Invisible to JS — and excluded entirely by synthetic-event benchmarks (§1.3.3).
2. **EventPath construction.** Per dispatch, the engine computes "a static ordered list of all its ancestors in tree order," once, up front.[3] Cost is O(path length); deep trees pay more before any listener is consulted.
3. **Per-node listener lookup.** For each object on the path, the engine scans its listener list for type and capture match, plus once/removed/passive bookkeeping, before user code runs.[3] [7] Long listener lists on hot nodes multiply fixed dispatch overhead.
4. **Per-listener C++→JS invocation.** Each matched listener crosses the engine boundary: arguments marshaled, `currentTarget` set, invoke-loop bookkeeping applied. This is where N leaf listeners lose to one delegated listener.[2]
5. **Handler body.** Your code — and keyboard events make it uniquely expensive: keyboard default actions run synchronously on the main thread after dispatch, so a slow `keydown` handler *directly* delays text insertion, scrolling, and focus movement for that keystroke.

Layers 1–4 are fixed per dispatch; layer 5 is where optimization lives. The vocabulary of §1.2–§1.4 is the toolset for shaving layers 3–5.

1.2 Event flow semantics that cost money

1.2.1 Capture → target → bubble; one path, computed once; flag semantics

The dispatch algorithm (WHATWG DOM): set the dispatch flag; initialize `target`; compute the event path once as a static ordered list of ancestors; run `CAPTURING_PHASE` down the path; run `AT_TARGET` listeners; if `bubbles`, reverse the path and run `BUBBLING_PHASE`; then clear flags, reset `eventPhase` to `NONE` and `currentTarget` to `null`.[3] [8] The path is *not* recomputed if handlers mutate the DOM mid-dispatch — "once determined, the propagation path must not be changed… even if an element in the propagation path is moved within the DOM or removed from the DOM."[9] Mutating the tree inside a handler never saves dispatch work for the current event.

`composedPath()` returns that precomputed path (target → … → `window`), including shadow-tree nodes for open roots and omitting closed roots.[10] [11] The path already exists internally, so the call is essentially a materialization — but it **allocates a fresh JS array on every call**. On a hot key path, prefer `event.target` (already retargeted).

Flow control is flag bookkeeping, and each method buys back a different slice of layers 3–4:

- `stopPropagation()` sets the stop propagation flag; remaining listeners *on the current node still run*, but nodes further along the path are skipped.[3]
- `stopImmediatePropagation()` sets both stop flags; even sibling listeners on the current node are skipped — the cheapest way to truncate dispatch.[3] [7]
- `preventDefault()` sets the canceled flag iff `cancelable` is true *and* the in-passive-listener flag is unset.[3] It does **not** stop propagation — and vice versa. Swallowing a key fully requires both.

1.2.2 Passive defaults, cancelability, retargeting and `composed`

Passive-by-default is a scroll intervention, not a keyboard one: in browsers other than Safari, `passive` defaults to `true` for `wheel`, `mousewheel`, `touchstart`, and `touchmove` listeners registered at the document/window level.[2] [12] Keyboard events are **never** defaulted to passive anywhere — not an omission: keyboard default actions are not compositor-driven scrolls but run synchronously on the main thread after dispatch, so there is nothing for a passive contract to unblock. Your `keydown` handler is always on the critical path.

Cancelability is per-type: `keydown`, `keyup`, `keypress`, `beforeinput` (outside IME internals), and `compositionstart` are cancelable; `input`, `compositionupdate`, and `compositionend` are not, per spec — with Chrome/Safari deviating on the composition events.[13] [14]

For shadow DOM, two facts suffice. Retargeting: as an event crosses a shadow boundary, each listener sees `event.target` rewritten to "an element in the same scope as the listening element"; `composedPath()[0]` recovers the original target.[10] [11] `composed`: keyboard events are `composed: true` (as are `beforeinput`, `input`, the four focus events, and all composition events), while custom events default to `composed: false` — "a `bubbles`-only custom event silently dies at the shadow root."[11] [15] Because `keydown` is both composed and bubbling, one `document`-level listener sees keys from inside every shadow tree; per-component key listeners inside components mean handling the *same* event twice.

1.3 The KeyboardEvent surface

1.3.1 `key` vs `code` vs legacy; `location`; `repeat`; `isComposing`; modifiers; `getLayoutMap`; special values

Property	Type	Semantics	Perf-relevant note
key	string	Meaning after layout + modifiers: "q"/"Q", "Enter", "Dead", "Process", "Unidentified"; falls back to "Unidentified" when nothing can be determined.[16]	Requires engine layout resolution; layout-dependent (Control+Q → "q" on US).
code	string	Physical position, layout-independent: "KeyA" is "labelled q on an AZERTY keyboard."[17]	The right key for positional shortcuts; stable interned-string compare.
keyCode / charCode / which	number	**Deprecated**; "a system and implementation dependent numerical code… inconsistent across platforms";[18] [13] which is the Netscape-legacy union.[19]	Ubiquitous in legacy code; number compare is fast but layout-dependent. IME marker: **229**.[20]
location	number	STANDARD=0, LEFT=1, RIGHT=2, NUMPAD=3.[13] [21]	Cheap constant compare; code ("ShiftLeft") often

Property	Type	Semantics	Perf-relevant note
			already encodes it.
repeat	boolean	true once the key triggers key repetition.[13]	First-line filter: if (e.repeat) return; — but see Firefox/X11 bugs.[22]
isComposing	boolean	true while the event belongs to an IME composition session.[13]	One boolean read; the standard IME guard.
altKey/ctrlKey/ metaKey/shiftKey	boolean	The four classic modifiers.	Cheapest possible reads; sufficient for most shortcut logic.
getModifierState(k)	method	Any modifier — "AltGraph", "CapsLock", "Fn", "OS", plus virtual "Accel" (Control on Windows, Meta on macOS).[23] [24]	The only way to read lock states; a call + string compare, so cache when scanning many events.
KeyboardEvent.get LayoutMap()	static → Promise	code → label map for the highest-priority ASCII-capable layout; secure-context, top-level-only, Permissions-Policy gated.[25]	**Async** — unusable inside a handler for *that* keystroke; cache at startup, listen for layoutchange. Chromium-only in practice.[25]

The three special `key` values are correctness traps with cost consequences: `"Dead"` for dead keys, `"Process"` for keys consumed by IME processing, `"Unidentified"` when no value can be determined (virtual keyboards often report keyCode 229 + `"Unidentified"`).[16] [20] Shortcut matchers that do not exclude these will treat IME input as commands. Note also the constructor asymmetry: `new KeyboardEvent(type, init)` accepts `key`, `code`, `location`, `repeat`, `isComposing`, and modifier inits — but **`keyCode`/`charCode`/`which` are not settable** and stay 0 on synthetic events.[13] [26]

1.3.2 Ordering, keydown cancellation, auto-repeat, 229

One printable character produces, in shipping order (Chrome/Safari, effectively all modern engines): `keydown` → `keypress` → `beforeinput` → `input` → `keyup` — the draft spec orders `beforeinput` before `keypress`, a documented mismatch (uievents #220).[27] `keypress` is deprecated legacy, fired only for character-producing keys.[13] Cancellation is asymmetric in a way shortcut authors must internalize:

"Canceling the default action of a `keydown` event MUST NOT affect its respective `keyup` event, but it MUST prevent the respective `beforeinput` and `input` (and `keypress` if supported) events from being generated."[13]

So canceling `keydown` suppresses character insertion, arrow/Space/PageUp scrolling,[28] and Tab focus movement — the exact mechanism focus-trap libraries use for containment[29] — while modifier bookkeeping still occurs ("the keystroke MUST still be taken into account for

the modifiers states").[13] Dead keys and IME sequences can be canceled only on `keydown`; "canceling a dead key on a `keyup` event has no effect."[13] Browser-chrome shortcuts (Ctrl+T et al.) are generally not cancelable from page JS at all.

Auto-repeat has a defined shape: a held key produces repeating `keydown` → `beforeinput` → `input` triples "at a rate determined by the system configuration," with `repeat: true` on the repeated keydowns;[13] `keyup` fires once on release (whether `keyup.repeat` may be true is an open question, uievents #396).[30] Divergence: Firefox under X11/XINPUT2 can emit repeated keydowns with `repeat: false`,[22] and on mobile long-press the first `repeat: true` event signals the long press itself.[13] [20] During IME processing, legacy `keyCode` is conventionally 229 — the de-facto "this keystroke is not yours" marker.[20]

1.3.3 Trusted vs synthetic: the benchmarking trap

`el.dispatchEvent(new KeyboardEvent('keydown', {key:'a', code:'KeyA'}))` runs the *full* dispatch algorithm — path construction, phases, listener invocation — so layers 2–4 of §1.1.2 are faithfully exercised. Everything else is absent: `isTrusted` is false ("true when the event was generated by a user action, and false when… dispatched via dispatchEvent"[31]); **no default actions occur** — no insertion, scrolling, focus move, or `beforeinput/input` cascade ("we can either dispatch the correct events OR emulate typing, not both"[32]); legacy `keyCode` reports 0;[26] and synthetic keydown does not chain into keypress/beforeinput/input automatically.

```
// TRAP: this measures dispatch + listener cost ONLY —
// no layout mapping, no IME, no editor default actions, no paint.
const t0 = performance.now();
for (let i = 0; i < N; i++) el.dispatchEvent(new
KeyboardEvent('keydown', {key:'a'}));
// For trusted-input numbers you need OS-level automation
// (WebDriver / CDP Input.dispatchKeyEvent), not dispatchEvent.
```

Every `dispatchEvent` microbenchmark in the wild undercounts the real cost of a keystroke by exactly layers 1 and 5's default-action tail.

1.4 Focus, targeting, and IME

1.4.1 The keyboard target rule; `focus/blur` vs `focusin/focusout`

Trusted key events originate at "the focused element processing the key event or if no element focused, then the body element if available, otherwise the root element."[13] When the focused element is removed, disabled, or hidden, the HTML focus fixup rule moves the focused area to the viewport, "reflected through the `activeElement` API as the body element."[33] [34] The event then bubbles `element` → … → `body` → `html` → `document` → `window`: a `window` keydown listener is the last bubble stop; a `window` *capture* listener is the first code to see the key. A document-level listener therefore needs no focus management to observe everything.

Focus transitions are observable through two pairs: `focus`/`blur` do **not** bubble; `focusin`/`focusout` do — and all four are `composed: true`.[11] [35] Capture listeners on ancestors are the alternative channel for the non-bubbling pair. This matters because shortcut scope is almost always focus scope: *which* pair you listen to determines whether one delegated handler suffices or per-element registration is required.

1.4.2 Composition sessions and cross-browser divergence

A composition session is exactly "one `compositionstart` event, one or more `compositionupdate` events, and one `compositionend` event."[13] Key events flow throughout: "during the composition session, `keydown` and `keyup` events MUST still be sent, and these events MUST have the `isComposing` attribute set to `true`."[13] The progression is precise: the initiating `keydown` has `isComposing: false`, everything inside the session has it `true`, and the committing `keydown` is followed by `compositionend`, then (per spec) `keyup`.[13] Input events inside a session run `beforeinput` → `compositionupdate` → DOM update → `input`; canceling `beforeinput` prevents the DOM update and the `input`; no `beforeinput`/`input` accompany `compositionend`.[13]

Interoperability diverges at the edges. The spec makes `compositionupdate`/`compositionend` non-cancelable; Chrome/Safari make them cancelable; Firefox makes even `compositionstart` non-cancelable, and Chrome/Safari additionally fire the legacy `textInput` between `beforeinput` and `input`.[27] Around session end, Firefox/Safari emit a `keyup` after `compositionend` while Chrome/Edge do not, and Safari emits an extra `keydown` with `which === 229`.[36] Engine bugs surface too — Chromium 147 fired a spurious `deleteContentBackward` before `compositionstart`.[37] The practical handler rule is also the cheap one: skip `keydown` handling while `e.isComposing` (or `e.keyCode === 229`), and treat the post-`compositionend` `keyup` as possibly swallowed, per-browser. One boolean read buys immunity to the entire divergence surface:

```
function onKeydown(e) {
  if (e.repeat || e.isComposing || e.keyCode === 229) return; // one-line IME/repeat guard
  // ... shortcut logic
}
```

2. Engine Internals: The Dispatch Machine

Chapter 1 modelled a keystroke as five semantic cost layers: input delivery, path construction, listener lookup, invocation, attribute access. This chapter grounds each layer in source. The established architecture — Blink's `EventDispatcher`, WebKit's `EventDispatcher.cpp`, Gecko's `EventTargetChain` — has been stable since the WebKit/Blink fork; the recent changes are optimizations bolted on (SkipEventCapture, passive interventions) and instrumentation wrapped around it (Event Timing / INP). Stable machine first, then the deltas.

2.1 The Blink input pipeline for real keys

2.1.1 Browser process → InputRouter → compositor → main thread

A physical key press enters Chromium as an OS-native message, converted to a platform-independent `ui::Event` / `NativeWebKeyboardEvent` in the browser process; IME interaction happens here too [38]. The browser-side funnel, `RenderWidgetHostImpl::ForwardKeyboardEventWithCommands` (content/browser/renderer_host/render_widget_host_impl.cc), is not a passive relay: `KeyPressListenersHandleEvent` serves browser keypress listeners and `delegate_->PreHandleKeyboardEvent` serves accelerators — "Tab switching/closing accelerators aren't sent to the renderer to avoid a hung/malicious renderer from interfering" [39]. A browser-consumed RawKeyDown also suppresses its subsequent Char/KeyUp via `suppress_events_until_keydown_` [39].

Delivery is asynchronous: the async `InputRouter` with ACKs replaced the old sync-IPC path, per the design principle that "keyboard events should not use synchronous IPC calls" [40] [41]. In the renderer, `InputHandlerProxy` runs on the compositor thread deciding what can be handled off-main-thread — wheel events with passive/no blocking listeners are marked `DID_NOT_HANDLE_NON_BLOCKING` or `DROP_EVENT` without touching the main thread [42]. **Keyboard has no such fast path.** The entirety of `InputHandlerProxy`'s keyboard handling is frame attribution:

```
if (WebInputEvent::IsKeyboardEventType(event.GetType())) {
  // Keyboard events should be dispatched to the focused frame.
  return
WebInputEventAttribution(WebInputEventAttribution::kFocusedFrame);
}
```

[42] Keys are always forwarded to the main thread as blocking input. Consequences: key-driven scrolling (arrows, space, PageUp) is main-thread work inside Blink's `KeyboardEventManager` / `ScrollManager::LogicalScroll` default handlers, unlike compositor-eligible wheel/touch scrolling [42]; the "[Violation] non-passive listener" and "input event was delayed" console machinery applies to scroll-blocking wheel/touch only — keys get neither passive defaults nor blocked-event warnings [43]; and while `compositor_event_queue_->CoalesceEvents()` merges continuous streams (scroll, pinch, mousemove), discrete key events are **never coalesced and never frame-aligned** (`DeliverInputForBeginFrame` applies to the continuous queue only) [42].

2.1.2 Auto-repeat: one full pipeline traversal per tick

OS auto-repeat generates one RawKeyDown/KeyDown (+Char) per tick, with no dedup or throttling in Blink: each repeat is a new `WebKeyboardEvent` carrying the `kIsAutoRepeat` modifier, a new `KeyboardEvent`, a full dispatch [42] [44]. The `repeat` getter is a pure bit test —

```
bool repeat() const { return GetModifiers() &
WebInputEvent::kIsAutoRepeat; }
```

[44] — the engine hands you the flag to filter repeats with, and nothing else. At a typical OS auto-repeat rate (~25–30 repeats/second, user-configurable) a held key is one complete trip through §2.1–§2.4 per repeat; debouncing belongs in the library.

On the main thread, `EventHandler::KeyEvent` delegates to `KeyboardEventManager::KeyEvent` (core/input/keyboard_event_manager.cc) [45] [46]: resolve the focus target (`EventTargetNodeForDocument`, body/document fallback — re-resolved after keydown before synthesizing keypress, since "Focus may have changed during keydown handling"), fire user-activation notification for non-modifier keys, construct a `KeyboardEvent` per phase, call `node->DispatchEvent(*event)`, and run default handlers (access keys, scroll keys, focus navigation) if not canceled [45]. Note the multiplication: a character-producing keystroke is **two full dispatch passes** — keydown via RawKeyDown/KeyDown, keypress via Char — each building an EventPath and walking listeners [45].

2.2 The dispatch algorithm's cost structure

2.2.1 EventPath: O(depth), exactly-sized, never cached

`EventDispatcher::DispatchEvent` begins by constructing an `EventDispatcher`, whose constructor calls `event_->InitEventPath(*node_)` [47]. `EventPath::CalculatePath()` walks from the target to the root, handling assigned slots and shadow-root retargeting (`GetShadowRootParent` respects `event.composed`), gathering nodes on the stack before materializing the result [48]:

```
// For performance and memory usage reasons we want to store the
// path using as few bytes as possible and with as few allocations
// as possible which is why we gather the data on the stack before
// storing it in a perfectly sized node_event_contexts_ Vector.
HeapVector<Member<Node>, 64> nodes_in_path;
...
node_event_contexts_ = HeapVector<NodeEventContext>(nodes_in_path,
...);
```

[48] Machine-level reading: a stack-resident `HeapVector<Member<Node>, 64>` accumulates up to 64 pointers inline (typical depths fit without spill); one GC-heap allocation of exactly `depth × sizeof(NodeEventContext)` copies them out; `CalculateAdjustedTargets()` adds `TreeScopeEventContext` objects per TreeScope crossed for shadow retargeting [48]. There is **no path cache** — the DOM may have mutated between events, so every dispatch pays O(D) pointer-chasing up the ancestor chain (realistically one cache miss per uncached ancestor), one variable-length heap allocation, and the attendant GC tracing. Depth is the fixed cost of every keydown, paid even when zero listeners exist.

Dispatch proper is the textbook three-phase walk [47]:

```
if (DispatchEventLegacyPreActivationBehavior(...) ==
kContinueDispatching) {
  if (DispatchEventAtCapturing() == kContinueDispatching) {
    DispatchEventAtBubbling();
  }
}
DispatchEventPostProcess(activation_target, ...);
```

Capture walks the path in reverse, bubble forward; each `NodeEventContext::HandleLocalEvents` sets the shadow-adjusted `target`, `currentTarget`, and `invocationTargetInShadowTree` before calling `node_->HandleLocalEvents(event)` [49]. `stopPropagation()`/`stopImmediatePropagation()` flags are re-checked after every node and listener — O(1) per check, but constant. After bubbling, `DispatchEventPostProcess` runs default handlers in bubbling order for trusted events — where scroll-on-space, accesskey activation, and editor/IME handling hook in [47]. `preventDefault()` is thus cheap for the engine: it sets the cancel bit this stage consults; nothing already delivered is unwound.

2.2.2 SkipEventCapture: one stray listener disables it page-wide

The significant recent change to this architecture is SkipEventCapture, enabled by default: `DispatchEventAtCapturing()` early-returns when `!node_->GetDocument().HasCaptureListener()`, gated by `RuntimeEnabledFeatures::SkipEventCaptureEnabled()` [47] — "Improves performance of event dispatching by skipping the capture phase if there are no capture listeners registered on the page" [50]. On a listener-free page a keydown costs roughly one ancestor walk instead of two. The predicate is **document-global**: one `addEventListener(type, fn, true)` anywhere — including in a third-party widget you don't control — flips `HasCaptureListener()` and restores the full capture walk for *every* dispatch on the page. WebKit implements the same optimization independently via per-document `EventListenerCounts` (`listenerCounts.hasCapturing()`); its `EventDispatcher.cpp` otherwise mirrors Blink (common ancestry) — `dispatchEventInDOM` walks the path in reverse for capture, forward for bubble, `callDefaultEventHandlersInBubblingOrder` runs defaults [51].

2.2.3 Per-node lookup and the spec-mandated snapshot

At each path node, `Node::HandleLocalEvents` early-returns if the node has no `EventTargetData` — a lazily allocated side table, so listener-less nodes cost one null check [52]. Otherwise `EventTarget::FireEventListeners` calls `d->event_listener_map.Find(event.type())` [43]. `EventListenerMap` is a linear vector of (AtomicString type → EventListenerVector) pairs; `Find` scans the *distinct types registered on that node* — cheap, since per-node type counts are tiny and AtomicString comparison is a pointer compare after interning [53]. Lookup thus scales with distinct types on the node, not listeners page-wide.

The firing loop then pays the spec-mandated copy-on-fire:

```
// Fire event listeners, creates a copy of EventListenerVector on
being called.
bool EventTarget::FireEventListeners(Event& event, EventTargetData* d,
                                     EventListenerVectorSnapshot
entry)
```

with `using EventListenerVectorSnapshot = HeapVector<Member<RegisteredEventListener>, 1>;` [43] [54]. The header is explicit: "This method makes a copy of the `EventListenerVector` on invocation to match the HTML spec. Do not try to optimize it away." [54] Every node with ≥1 listener of the type pays one vector clone per dispatch — inline capacity is 1, so two or more listeners spill to the GC heap.

Per listener in the snapshot [43]:

```
if (registered_listener->Removed()) [[unlikely]] continue;
if (event.ImmediatePropagationStopped()) break;
if (!registered_listener->ShouldFire(event)) continue;   //
capture/bubble phase filter
EventListener* listener = registered_listener->Callback();
if (registered_listener->Once()) {
  removeEventListener(event.type(), listener, registered_listener-
>Capture());
}
event.SetHandlingPassive(EventPassiveMode(*registered_listener));
probe::UserCallback probe(context, nullptr, event.type(), false,
this);
listener->Invoke(context, &event);
```

Three details matter for hot paths. (1) `ShouldFire` is the per-listener capture/bubble phase filter over bitfield-packed options in `RegisteredEventListener` [55] — a wrong-phase listener still occupies the snapshot and pays its checks. (2) `once: true` is removeEventListener *immediately before Invoke* — an extra `EventListenerMap::Remove` scan + O(listeners) vector erase at fire time [43] [53]. (3) every invocation is bracketed by `probe::UserCallback` DevTools instrumentation [43]. Registration-side: `addEventListener` dedups on (callback, capture) only — passive/once/signal are ignored for matching [53] — and since crbug.com/1420890, `EventListenerMap::Add` annotates crash keys with `listener_count_log2` past 8 same-type listeners on one node [53].

The assembled model for an event bubbling through D ancestors, node i holding L_i listeners of the type: O(D) path build + one GC allocation, plus per node-with-listeners one snapshot clone ($O(L_i)$ alloc+copy), plus ΣL_i invocations each crossing into V8. A single delegated `document` keydown listener is the minimal shape: O(D) path + 1 clone + 1 crossing.

2.3 KeyboardEvent getter internals

2.3.1 Blink: eager construction, cached reads

Blink computes `key` and `code` **eagerly in the constructor** and caches them [44]:

```
KeyboardEvent::KeyboardEvent(const WebKeyboardEvent& key, ...)
    : ...
      // TODO(crbug.com/482880): Fix this initialization to lazy
initialization.
      code_(FromUtf8(ui::KeycodeConverter::DomCodeToCodeString(
          static_cast<ui::DomCode>(key.dom_code)))),
      key_(FromUtf8(

ui::KeycodeConverter::DomKeyToKeyString(ui::DomKey(key.dom_key)))),
```

with trivial getters `const String& code() const { return code_; }` / `key()` [44]. The mapping machinery (ui/events/keycodes/dom/keycode_converter.cc) runs once per event: `DomCodeToCodeString` generates runs ("KeyA"..."KeyZ", "Digit0..9", "F1..F24") arithmetically, else linearly scans `kDomCodeMappings`; `DomKeyToKeyString` collapses dead keys to "Dead", scans `kDomKeyMappings`, else UTF-8-encodes the character [56] — whether or not JS ever reads the strings, hence crbug.com/482880's standing TODO to make it lazy. Per-read cost in a listener is just the V8 attribute-getter callback (a binding call) plus wrapping the cached, shared `WTF::String`: no re-atomization, no table lookup. `keyCode`/`charCode`/`which` are plain unsigned fields set at construction; `location` derives from modifier bits; `repeat` is the bit test above [44]. WebKit shares the compute-once shape — `m_key(key.key())`, `m_code(key.code())` cached from `PlatformKeyboardEvent`, native codes mapped through the `PlatformEventFactory{Mac,Win,Gtk}` tables (WebKit bug 149584) [51] [57].

2.3.2 Gecko: re-mapped per access

Gecko differs materially: `KeyboardEvent::Key()` / `KeyboardEvent::Code()` call `WidgetKeyboardEvent::GetDOMKeyName()` / `GetDOMCodeName()` **on each access**, mapping `mKeyNameIndex`/`mCodeNameIndex` through key-name string tables into a caller-provided nsString rather than caching (ResistFingerprinting can spoof `Code` per access) [58]. Its dispatch core (`dom/events/EventDispatcher.cpp`) builds an `EventTargetChain` of fixed-capacity `EventTargetChainItem` objects with per-item `MayHaveListenerManager` flags to skip listener-less nodes [59]. In Firefox, repeated `event.key` reads in a hot loop do real string work per read: hoist `e.key` into a local — near-free in Blink/WebKit, a genuine win in Gecko.

Layer	Blink (Chromium)	WebKit	Gecko
Path structure	HeapVector<NodeEventContext>, exactly-sized, rebuilt per dispatch [48]	EventPath, mirrors Blink [51]	EventTargetChain of fixed-capacity items [59]
Capture-	HasCaptureListener() +	Document::EventList	per-item

Layer	Blink (Chromium)	WebKit	Gecko
skip	SkipEventCapture, default-on [47] [50]	enerCounts::hasCapturing() [51]	MayHaveListenerManager skip [59]
Listener snapshot	copy-on-fire EventListenerVectorSnapshot per node [54]	equivalent (shared ancestry) [51]	chain items + listener manager
key/code	eager at construction, cached [44]	eager at construction, cached [51]	table-mapped per getter call [58]
Key fast path off main thread	none; keys always main-thread [42]	—	—

2.4 V8 invocation anatomy

Per listener firing, Blink executes the full `JSBasedEventListener::Invoke` spine (bindings/core/v8/js_based_event_listener.cc): bail-out checks (execution terminating, world mismatch); `HandleScope`; `GetListenerObject(currentTarget)` (may lazily compile a content-attribute handler); `ScriptState::Scope` entering the listener's context; `probe::InvokeEventHandler`; the `BindingSecurity::ShouldAllowAccessToV8Context` check; event wrapper materialization via `ToV8Traits<Event>::ToV8` [60]. Per the V8 Binding Design doc, a C++ object exposes *the same* JS wrapper per world — cached on `ScriptWrappable::main_world_wrapper_` for the main world, in `DOMWrapperMap` otherwise [61] — so the first listener allocates the JS `KeyboardEvent` wrapper and same-world successors hit the cache: one wrapper per event per world, not per listener. Then `window->SetCurrentEvent(event)` bookkeeping and a `v8::TryCatch` with `SetVerbose(true)` around the call [60].

`JSEventListener::InvokeInternal` (js_event_listener.cc) resolves the callable: a plain function is a direct `v8::Function::Call`, but an object listener goes through `GetEffectiveFunction`, a **property Get("handleEvent") per invocation** [62]. `V8ScriptRunner::CallFunction` (v8_script_runner.cc) adds a recursion-depth check, a `V8RunMicrotasksScope` (microtask checkpoint on scope exit for the outermost call), `probe::CallFunction`, and `function->Call` → V8 `Execution::Call` [63]; an older stack crawl shows the identical spine — `FireEventListeners → V8AbstractEventListener::handleEvent → InvokeEventHandler → V8EventListener::CallListenerFunction → V8ScriptRunner::CallFunction → v8::Function::Call → Execution::Call` [64].

Hence N listeners = N boundary crossings with no possible batching: every step above runs per listener — HandleScope, context scope, security check, TryCatch, probes, the CEntry trampoline — and since the listener loop lives in C++ (`FireEventListeners`), control ping-pongs C+

+→JS→C++ N times per dispatch. Fewer, fatter listeners that switch on `e.code` in JS amortize this; many tiny natively-registered ones do not. Attribute reads inside the handler are further binding callbacks *from* JS into C++ — individually cheap (cached fields, §2.3) but never plain JS property loads. Teardown has the same shape: `AbortSignal` removal has no batch fast path — each signal-registered listener adds a separate abort algorithm calling plain `removeEventListener`, tracked as (listener → AlgorithmHandle) pairs in the AbortSignalRegistry (spec: whatwg/dom#911 / PR #919), so `controller.abort()` over K listeners costs K individual map scans [43] [65].

2.5 Where per-keystroke latency actually lives

End-to-end, a real key press decomposes into: OS/UI-thread processing (IME, accelerators) [38]; browser-side filtering [39]; async IPC through InputRouter to the compositor thread, which forwards keys straight to the main-thread task queue [42] [66]; **waiting for main-thread availability** — the dominant variable term: input tasks are high-priority but cannot preempt a running task, and timers/postMessage visibly starve under continuous key input on a saturated main thread (Chromium closed one such report as an intentional scheduling tradeoff [67]); then `KeyboardEventManager` construction + dispatch per §2.2–2.4 [45] [47]; then any style/layout/paint fallout on the next frame. Synthetic `dispatchEvent(new KeyboardEvent(...))` skips the pipeline stages but traverses identical dispatch machinery — its microbenchmarks measure §2.2/§2.4 only.

Every dispatch is instrumented: `EventDispatcher::Dispatch` constructs a `UIEventTiming event_timing(frame, *event_)` recording processing start/end for the Event Timing API — the source of `processingStart`/`processingEnd` and INP — and keyboard events are always eligible (`IsStandardEventType` includes `IsA<KeyboardEvent>`) [68]. `event.timeStamp` equals the Event Timing `startTime`, so page code can correlate its own measurements with the engine's [69]; Gecko mirrors this via `PerformanceEventTiming::TryGenerateEventTiming` per event [59].

Change	Where	Effect
SkipEventCapture (default-on)	DispatchEventAtCapturing, event_dispatcher.cc [47] [50]	Skips capture walk when document has no capturing listeners; one stray capture listener disables it page-wide
crbug.com/482880 (TODO)	KeyboardEvent ctor [44]	key_/code_ eager; every keydown pays the mapping even if unread
crbug.com/1420890	EventListenerMap::Add [53]	listener_count_log2 crash keys past 8 same-type listeners/node
Passive-by-default interventions (M56 touch, M73 wheel)	SetDefaultAddEventListenerOptions [43] [70]	Keyboard deliberately excluded; passive is a no-op on key events

Change	Where	Effect
WebKit bug 188370	EventDispatcher resetBeforeDispatch [71]	IME-handled keys must not run default handlers

This architecture yields testable predictions for Chapter 3. (1) Dispatch cost should scale with DOM **depth**, not with the listener's position in the path — a delegated `document` listener pays O(D) path work regardless of tree shape. (2) Capture and bubble dispatch should cost the same on a page with any capturing listener, and drop ~one traversal when the document has none (SkipEventCapture). (3) Repeated `e.key/e.code` reads should be near-free in Blink/WebKit (binding call only) but measurably costlier per read in Gecko. (4) `{handleEvent}` object listeners should be slightly, consistently slower than plain functions (the per-invocation Get). (5) `preventDefault()` should be near-free — a flag set, nothing unwound. (6) Listener-count scaling should be strictly linear in boundary crossings, and synthetic-dispatch benchmarks should undershoot real-key latency by exactly the input-pipeline and main-thread-wait terms.

3. Measured Results: Chromium 150 Sandbox

Every number in this chapter was measured in this investigation's sandbox: **Chromium 150.0.7871.181** (real Blink + V8, headless, `--no-sandbox --disable-gpu --disable-dev-shm-usage --js-flags=--expose-gc`), driven by Node v20.20.2 on 2026-08-13. Unless stated otherwise, figures are median ops/s over 2 full runs (10 pooled repetitions of ≥300 ms measured time each, batches calibrated to ≥2 ms, ~200 ms warmup per case, alternating case order, `gc()` between suites, and a DCE sink guard whose final value was verified nonzero); CV is stdev/mean across pooled reps. Two outlier cases were rerun once and pooled (15 reps) and are flagged where they appear. Sections 3.1–3.5.1 are the **engine-cost** measurements — synthetic `dispatchEvent` on the Blink main thread, no IPC and no input pipeline. Section 3.5.2 reports the trusted layer, which is driver-bound and must be read differently. Absolute numbers are machine-specific; the ratios are the portable result.

3.1 Dispatch topology

3.1.1 Listener position is irrelevant; path length is everything

Table 3-1. Keydown dispatch + one listener vs. tree depth and listener position (detached trees, so path length == depth; median ops/s, n=10 pooled reps × 2 runs, Chromium 150 headless)

case	median ops/s	ns/op	CV	rel. to fastest
leaf-listener depth=1	252.1 k	3,967	4.0 %	0.998
root-listener depth=1	252.6 k	3,958	3.3 %	1.000
leaf-listener depth=50	154.3 k	6,481	4.0 %	0.611
root-listener depth=50	142.3 k	7,027	5.9	0.563

case	median ops/s	ns/op	CV	rel. to fastest
leaf-listener depth=500	31.5 k	31,725	5.1 %	0.125
root-listener depth=500	31.3 k	31,968	9.2 %	0.124

Chapter 2 predicted that dispatch cost scales with EventPath length rather than listener position; the data **confirms** both halves. At depth 1, leaf and root listeners run at 252.1k vs 252.6k ops/s — a ratio of 1.002, statistically indistinguishable. Deepen the tree and the two columns collapse together: at depth 500, leaf and root land at 31.5k and 31.3k ops/s (~32 µs per dispatch), a 1.006 ratio. Meanwhile the depth scaling is severe: depth 1→50 costs 1.6–1.8× (252.6k→142.3k ops/s for the root case, 1.78×), and depth 1→500 costs ~8×. Blink computes the propagation path once per dispatch and walks it regardless of where handlers sit, so a handler at the root of a 500-node tree pays the same path-traversal tax as one on the leaf. Note the depth-500 root case carries the highest CV in the suite (9.2%), typical of the longest per-op times; the leaf/root agreement nonetheless holds within run-to-run noise. Practical corollary: moving a listener "closer" to the target buys nothing unless it shortens the path actually traversed.

3.1.2 Capture ≈ bubble; same-node listeners are free, cross-ancestor listeners are not

Table 3-2. Capture vs. bubble at depth 500 (median ops/s, n=10; bubble@leaf pooled to n=15 after a flagged rerun)

case	median ops/s	ns/op	CV	rel. to fastest
bubble@leaf	30.9 k	32,374	9.9%	1.000
capture@root	28.9 k	34,639	7.3%	0.935
bubble@root	28.6 k	35,019	6.6%	0.924

Table 3-3. Listener count and placement (median ops/s, n=10)

case	median ops/s	ns/op	CV	rel. to fastest
100 listeners on target	248.4 k	4,025	2.2 %	1.000
1 listener on target	246.3 k	4,060	2.8 %	0.991
10 listeners on target	243.0 k	4,115	3.2 %	0.978
10 listeners on target (depth10 tree)	221.7 k	4,510	2.4 %	0.892
10 listeners / 10 ancestors (depth10)	81.0 k	12,352	3.5 %	0.326

Read Table 3-2 in the context of Table 3-1: every capture/bubble permutation at depth 500 clusters at 28.6–30.9k ops/s, the same ~32 µs regime as the depth-500 leaf/root cases, while the depth-1 baseline runs at ~252k. The phase in which a listener fires contributes nothing measurable next to the 500-entry path walk — the ns/op spread between the fastest and slowest row is 2.6 µs against a 10.6 µs spread across bubble@leaf's own reps. If capture were to carry an intrinsic cost, this is the configuration (longest path, earliest-phase listener) where it would have to appear; it does not. Chapter 2's second prediction — capture ≈ bubble — is **confirmed**: at depth 500, capture@root (28.9k ops/s) and bubble@root (28.6k) sit at a 1.01 ratio, within CV. Phase choice is free. One honest caveat: `bubble@leaf` showed a **bimodal distribution across runs** (~26k vs ~32k ops/s, a JIT/GC tier shift); it was rerun once, pooled to 15 reps, and the reported 30.9k median carries the suite's highest CV (9.9%). We flag rather than hide this. Table 3-3 sharpens the topology story: stacking 1, 10, or 100 listeners on the *same* node stays within 2.2% (246.3k / 243.0k / 248.4k ops/s — non-monotonic, hence noise), because the path is computed once and per-node listener invocation is cheap. But spreading those same 10 listeners across 10 ancestors drops throughput to 81.0k ops/s — 2.74× slower than 10 listeners on one node of the same depth-10 tree (221.7k). Dispatch is path-bound, not invocation-bound.

3.2 Delegation economics

3.2.1 Dispatch parity between delegated and direct listeners

Table 3-4. Delegated vs. direct dispatch (median ops/s, n=10)

case	median ops/s	ns/op	CV	rel. to fastest
delegated: 1 doc listener + closest()	176.1 k	5,677	3.5 %	1.000
direct: 1000 listeners on buttons	174.0 k	5,746	2.0 %	0.988
direct: 1 listener on target in 2000-node flat	148.4 k	6,737	3.1 %	0.843
delegated: 1 listener on 2000-node flat parent	147.0 k	6,805	3.8 %	0.834

Delegation's dispatch-time cost is a wash. One document-level listener doing `e.target.closest('[data-k]')` fires at 176.1k ops/s against 174.0k for 1000 direct per-button listeners (one firing per dispatch) — a 1.2% gap, inside the CV of either case. The same parity holds on a 2000-node flat tree (147.0k delegated vs 148.4k direct). The `closest()` call itself is not the bottleneck: a companion suite of delegated target checks found `closest` (127.9k ops/s), `matches` (125.7k), `tagName === 'BUTTON'` (125.1k), and `composedPath()[0]` (130.4k) all within 4.2% of each other, so the selection predicate can be chosen on correctness grounds alone. What the table hides is the setup side of the ledger — the 1000-listener column must first *register* 1000 listeners, which is where delegation actually wins, as the next section quantifies.

3.2.2 Registration is the real cost

Table 3-5. Registration throughput, delegated vs. direct (median ops/s, n=10)

case	median ops/s	ns/op	CV	rel. to fastest
addEventListener ×1 (delegated)	1.33 M	753	1.7 %	1.000
addEventListener ×1000 (direct)	1.3 k	766,871	2.0 %	0.001

A single `addEventListener` call costs ~0.75 µs (1.33M ops/s, CV 1.7% — one of the tightest measurements in the study). Registering 1000 direct listeners costs 766.9 µs per batch — almost exactly 1000× the single registration, i.e. perfectly linear with no batching discount in Blink's listener list management. This is the real delegation economics: at dispatch time the two architectures are at parity (Table 3-4), but the direct architecture pays three orders of magnitude more at setup and teardown. For a library mounting a keyboard-heavy widget with N focusable controls, delegated registration is O(1) in N; direct registration is O(N) at ~0.77 µs per control. The add+remove pair measured in §3.5.1 (744 ns) confirms registration and removal are symmetric in cost, so churning direct listeners on every render cycle multiplies this penalty.

3.3 Handler hot path

3.3.1 Property reads are all near-free

Table 3-6. Key-property reads inside the handler (median ops/s, n=10)

case	median ops/s	ns/op	CV	rel. to fastest
baseline (no read)	248.9 k	4,017	2.7 %	1.000
e.which	248.8 k	4,019	1.5 %	1.000
e.shiftKey	246.4 k	4,058	3.5 %	0.990
e.ctrlKey	245.5 k	4,073	3.5 %	0.986
e.code	244.7 k	4,087	2.9 %	0.983
e.charCode	241.9 k	4,135	2.2 %	0.972
e.key read twice	241.2 k	4,146	1.8 %	0.969
e.key	238.8 k	4,188	3.1	0.959

case	median ops/s	ns/op	CV	rel. to fastest
e.keyCode	238.5 k	4,194	3.6 %	0.958
e.getModifierState('Control')	233.7 k	4,278	2.1 %	0.939

Chapter 2's third prediction — near-free property reads — is **confirmed**. Every accessor, legacy (`keyCode`, `which`, `charCode`) or modern (`key`, `code`, modifier booleans, `getModifierState`), lands within 1–6% of the 248.9k ops/s no-read baseline, and the ordering is non-monotonic enough (`e.which` ties baseline at 1.000) that the deltas are mostly noise against CVs of 1.5–3.6%. Reading `e.key` twice costs the same as reading it once (241.2k vs 238.8k, within CV): V8 inlines the getter after the first call. Even the worst case, `getModifierState('Control')` — a real function call with a string argument — costs only ~6% (~260 ns per dispatch including dispatch itself). The practical rule: choose the property that is *correct* for your shortcut semantics (`code` for physical position, `key` for layout-resolved characters, deprecated aliases never for new code); there is no speed argument for any of them.

3.3.2 Shortcut matching: 7× between strategies

Table 3-7. Shortcut-matching strategy, handler-only throughput (no dispatch; median ops/s, n=10)

strategy	median ops/s	ns/op	CV	rel. to fastest
b) bitmask int + Map<number>	16.54 M	60	0.9 %	1.000
d) Map-of-Maps trie	14.58 M	69	18.6 %	0.881
a) normalized string + Map.get	5.38 M	186	3.1 %	0.325
c) linear scan of 50 bindings (hit last)	2.28 M	438	3.9 %	0.138

Strip dispatch away and the matching strategy dominates the handler: a bitmask integer keyed into a `Map<number>` sustains 16.54M ops/s (60 ns/lookup, CV 0.9%), essentially tied with a Map-of-Maps trie at 14.58M. The popular "normalized string" approach — `ctrl+shift+k` built via concat + `toLowerCase` then `Map.get` — runs 5.38M ops/s, **3.1× slower** than the bitmask; string construction, not the hash lookup, is the cost. A linear scan of 50 binding objects with the match placed *last* (worst case; average case ~2× faster) manages only 2.28M, **7.2× slower**. Caveat on the trie: one rep deoptimized (min 6.21M vs median 14.58M), inflating CV to 18.6% — flagged, rerun, and pooled; the median is robust but tail latency is not. Proportionality check: inside a real 4 µs synthetic dispatch, even the 438 ns scan is swamped —

these differences matter only for libraries doing thousands of matches per interaction (e.g., chord sequencers or per-keystroke palette filtering).

3.3.3 Guards are free; listener shape costs up to 10%

Table 3-8. Early-exit guards (median ops/s, n=10)

case	median ops/s	ns/op	CV	rel. to fastest
no guard	239.9 k	4,168	4.1 %	1.000
if (e.isComposing) return	239.3 k	4,179	3.3 %	0.997
if (e.repeat) return	238.9 k	4,185	4.4 %	0.996

Table 3-9. Listener registration shape (median ops/s, n=10)

case	median ops/s	ns/op	CV	rel. to fastest
function listener	248.3 k	4,027	2.6 %	1.000
arrow closure	243.3 k	4,110	2.8 %	0.980
onkeydown property	228.2 k	4,381	2.7 %	0.919
{handleEvent} object	223.9 k	4,466	2.4 %	0.902

Table 3-8's guards deserve a precision note: the deltas (−0.3% and −0.4%) are an order of magnitude smaller than the CVs (3.3–4.4%), so strictly the measurement can only bound the guard cost below ~0.5%, not prove it is exactly zero — a single boolean attribute read against a 4.2 µs dispatch is simply under the noise floor of this harness. That is the strongest form of "free" a microbenchmark can honestly claim, and it holds for both guards independently.

The two correctness guards every keyboard library should have — `e.repeat` to swallow auto-repeat and `e.isComposing` to stay out of IME composition — cost less than 0.5% (238.9k and 239.3k vs 239.9k ops/s baseline, deltas smaller than the ~4% CVs). There is no performance excuse for omitting either. Listener shape, by contrast, carries a small but real penalty. A plain function reference (248.3k ops/s) and an arrow closure (243.3k, −2%) are equivalent, but the `onkeydown` IDL property handler drops 8% (228.2k) and the `{handleEvent}` object form drops 10% (223.9k) — both stable across reps (CV ≤ 2.7%). The object form still earns its keep for stateful listeners carrying `this` without allocation, and the property form remains relevant for legacy interop; but for a hot global keymap, `addEventListener` with a stable function is measurably the cheapest shape.

3.4 Control calls, event types, construction

3.4.1 preventDefault within noise; stopPropagation is faster than propagating

Table 3-10. Event control calls (median ops/s, n=10)

case	median ops/s	ns/op	CV	rel. to fastest
no preventDefault	247.1 k	4,048	2.9 %	1.000
preventDefault()	243.5 k	4,107	4.1 %	0.986
stopPropagation() (2-level path)	235.3 k	4,250	3.7 %	0.952
stopImmediatePropagation (2 same-node)	232.9 k	4,294	3.6 %	0.943
2 listeners same node, no stopImmediate	243.1 k	4,114	2.0 %	0.984
no stopPropagation (2-level path)	201.6 k	4,960	1.9 %	0.816

Calling `preventDefault()` on a synthetic keydown is within noise of not calling it: 243.5k vs 247.1k ops/s (−1.5%, CV 4.1% on the treatment). On a detached, non-editable target there is no default action to cancel, so this measures only the flag-set; §3.5.2 shows the trusted side effects are a different (and favorable) story. `stopPropagation()` is the interesting case: on a 2-level path it runs at 235.3k ops/s versus 201.6k for letting the event propagate — **17% faster**, because it legitimately skips the ancestor listener invocation and the remaining path walk. Likewise `stopImmediatePropagation()` (232.9k) edges out two same-node listeners running to completion (243.1k) once its own early-exit saving is netted against the call overhead. The measured lesson for library authors: control calls are not overhead to avoid — `stopPropagation` in a handled-shortcut path is a genuine optimization, not merely a correctness device.

3.4.2 Event type changes dispatch cost by ~45%

Table 3-11. Construct + dispatch by event type (median ops/s, n=10)

event type	median ops/s	ns/op	CV	rel. to fastest
compositionstart (CompositionEvent)	359.3 k	2,783	2.9 %	1.000
beforeinput (InputEvent)	308.7 k	3,239	3.1 %	0.859
input (InputEvent)	308.1 k	3,24	3.0	0.858

event type	median ops/s	ns/op	CV	rel. to fastest
		6	%	
keyup (KeyboardEvent)	248.4 k	4,025	1.0 %	0.691
keydown (KeyboardEvent)	246.5 k	4,056	3.6 %	0.686

The event interface itself moves dispatch cost substantially. `compositionstart` constructs and dispatches at 359.3k ops/s; the two `InputEvent` types sit at ~308k; `keydown`/`keyup` trail at ~247k — KeyboardEvent is **~45% slower than CompositionEvent** (246.5k vs 359.3k) and ~25% slower than InputEvent, with keydown/keyup identical to each other (0.686 vs 0.691, keyup's CV a tight 1.0%). The extra cost lives in KeyboardEvent's wider init dictionary and Blink's per-event key/char code processing. Two caveats keep this honest: synthetic `beforeinput`/`input` on a detached, non-editable target triggers no editing behavior (the trusted layer in §3.5.2 does, which is why `preventDefault` changes trusted throughput), and these are construct+dispatch composites, so the deltas blend construction and path walk. For keyboard libraries the takeaway is asymmetric: your hot path is the *slowest* event type Blink offers — everything else in the input family is cheaper.

3.4.3 Event reuse is the single biggest synthetic-dispatch win

Table 3-12. Event construction and reuse (median ops/s, n=10)

case	median ops/s	ns/op	CV	rel. to fastest
dispatch, reused preconstructed event	775.9 k	1,289	9.8 %	1.000
construct only (no dispatch)	432.2 k	2,314	1.8 %	0.557
dispatch, fresh event each time	245.6 k	4,072	2.3 %	0.317

Decomposing the 4.1 µs fresh construct+dispatch baseline (245.6k ops/s): construction alone is 2.3 µs (432.2k ops/s) — over half the total — and dispatching a preconstructed, reused event collapses the operation to 1.3 µs (775.9k ops/s), a **3.2× speedup** over fresh construction. The reused case carries an elevated CV (9.8%; min 606.2k against a 789.3k max), reflecting GC pressure differences between the allocating and non-allocating variants, but the median gap is far outside that noise. This is the largest single lever found anywhere in the synthetic layer, and it comes with a hard boundary: only synthetic `dispatchEvent` flows can reuse event objects — trusted keyboard events are minted by the input pipeline and cannot be pooled. The technique is therefore a test-harness and synthetic-event-bus optimization (and a warning that fresh-construction microbenchmarks overstate dispatch cost by ~2×), not a production input-path optimization.

3.5 Registration/removal and the trusted layer

3.5.1 AbortController teardown is slower than explicit removal

Table 3-13. Registration and teardown strategies (median ops/s, n=10 — remove case pooled n=15 with agreement10=false, an outlier flagged per the preamble; teardown cases are full register+teardown cycles of 100 listeners)

case	median ops/s	ns/op	CV	rel. to fastest
add+remove pair	1.34 M	744	4.6%	1.000
once:true (register+dispatch+auto-remove)	183.3 k	5,456	4.7%	0.136
remove: 100 removeEventListener calls	22.6 k	44,160	23.0%	0.017
abort: 100 listeners removed via 1 abort()	11.9 k	83,815	11.1%	0.009

A single add/remove pair costs 0.74 µs (1.34M ops/s). `once: true` works fine for fire-once shortcuts: the full register+dispatch+auto-remove cycle runs at 183.3k ops/s, only ~34% above the plain dispatch baseline of ~4 µs — the auto-removal bookkeeping is modest. The surprise is teardown at scale. Removing 100 listeners via 100 explicit `removeEventListener` calls costs 44.2 µs per full cycle; the AbortController route — 100 signal-bound registrations plus one `abort()` — costs 83.8 µs, **1.9× slower end-to-end**. The signal bookkeeping at *registration* time outweighs the single-call removal convenience. Both teardown cases are the noisiest in the study (CV 23.0% and 11.1%; the remove case spans 9.0k–24.9k ops/s across reps), but the medians are separated by nearly 2×, well outside the noise. Verdict: use `AbortController` for ergonomics and leak-safety, not for speed.

3.5.2 The trusted layer: CDP-bound throughput and honest latencies

Table 3-14. Trusted key presses via CDP `Input.dispatchKeyEvent` (200 keydown→keyup presses per mode, key 'k' into a focused `<input>`, wall-clock from the Node driver; latency = `performance.now()` inside a capture-phase keydown handler minus a marker set immediately before dispatch, n=50 presses)

mode	presses/s (wall)	keydowns	beforeinput	input	avg latency (ms)	max latency (ms)
empty handler	252	200	200	200	1.27	6.6
shortcut handler (string match)	234	200	200	200	1.43	6.0
preventDefault on keydown	635	200	0	0	0.93	4.1

This table is framed differently from everything above it, deliberately. Sections 3.1–3.4 measured **engine cost** — Blink's EventPath computation and listener invocation on the main thread. The trusted layer measures the **input pipeline and the driver**: 234–252 presses/s

wall-clock is the CDP round-trip rate (Node→Chromium `Input.dispatchKeyEvent`), not anything V8 or Blink's event dispatch could influence, and trusted key events are scheduled through the input event queue (effectively frame/task-aligned), so per-press times are quantized and far noisier than synthetic numbers. The ~1.3–1.4 ms marker→handler latency *includes* CDP IPC and is an upper bound on true input-to-handler latency. Within those limits the table still yields two engine-relevant facts. First, `preventDefault()` on keydown **fully suppresses** `beforeinput` and `input` (200→0 events) — live confirmation of the keydown→beforeinput→input ordering and cancellation contract — and throughput rises to 635 presses/s because character insertion is skipped. Second, a real string-matching shortcut handler adds no measurable trusted-path latency over an empty handler (1.43 vs 1.27 ms, both dominated by IPC jitter with max latencies of 6+ ms), corroborating §3.3's synthetic finding that matching is cheap relative to everything around it.

The synthetic layer's ~4 µs dispatch and the trusted layer's ~1.3 ms marker-to-handler gap are the two ends of the latency budget that Chapter 4 assembles into a full key-press-to-paint accounting.

4. Published Evidence & the Latency Budget

4.1 The published microbenchmark record is thin

Anyone searching the literature for hard numbers on event-listener cost discovers, first, how little there is. The classic jsperf.com corpus is defunct and read-only, and the surviving harnesses (measurethat.net, its benchmarklab mirror, jsben.ch) host suites that mostly measure registration, not dispatch, and clicks, not keys. What follows is, to our knowledge, the complete published record with concrete archived numbers — plus the gaps, which are the more instructive half.

4.1.1 What exists, what it measured, and what nobody measured

Comparison	Published numbers	Scope / platform / year	Source
addEventListener('click', …) vs el.onclick	2,611 vs 2,650 exec/s — aEL ≈1.5% slower, effectively a wash	**Registration** cost only (not dispatch); browser unspecified; MeasureThat #32515, ~2023–2024	[72]
Vanilla click dispatch vs jQuery .click()	3,232,016 vs 1,283,495 ops/s — jQuery wrapper ≈2.5× overhead	Chrome 121, macOS 10.15.7, ~2024	[73]
CustomEvent dispatch vs plain callback (1,000 dispatches)	Callback wins by orders of magnitude; archived numbers vary per run	MeasureThat #20888; bundles allocation + dispatch + handler	[74]
onclick vs addEventListener	Suite exists, no archived aggregate numbers	jsben.ch	[75]
Capture vs bubble phase cost	*No published suite*	—	gap
Delegation vs N direct	*No published suite*;	—	[76]

Comparison	Published numbers	Scope / platform / year	Source
listeners (dispatch-time)	jasonformat (2020) calls the trade-off "hard to measure"		
passive:true vs passive:false dispatch cost	*No published suite*	—	gap
event.key vs keyCode vs which read cost	*No published suite*; jsben.ch "key access speed" measures Map/object lookup, not KeyboardEvent properties	—	[77]
Shortcut-matching strategies (string vs keyCode table vs bitmask)	*No published benchmark*; Mousetrap-vs-hotkeys comparisons cover features only	—	[78]

Three cautions attach even to the numbers that exist. Suite #32515 measures **registration**, and citing it as evidence about dispatch cost is a category error [72]. The jQuery comparison is real but tells you about wrapper layers, not the DOM event path [73]. And every synthetic-dispatch microbenchmark measures only dispatch+handler on a nested call stack — synthetic `dispatchEvent()` invokes handlers synchronously, excluding the input pipeline, IPC, and scheduling entirely [79] (MDN's "asynchronously via the event loop" phrasing is itself disputed; only dispatch is queued [80]). Add post-Spectre timer coarsening — Firefox ≥59 rounds `performance.now()` to 2 ms [81] — and single-dispatch sub-microsecond claims from any of these harnesses are noise.

The delegation debate deserves one more note. GreatFrontEnd (2021) asserts attaching 100 vs 10,000 listeners is "sub-millisecond" and that the real cost is memory — 10,000 retained closures versus one [82]. That is a practitioner claim with no numbers behind it; treat it as a hypothesis. Our sandbox in Chapter 3 filled most of these gaps directly: capture vs bubble, delegation economics at N = 1,000 listeners on a 2,000-node tree, and property-read costs are measured rather than asserted. Passive-on-keydown remains unmeasured in the sandbox — the relevant evidence is Akulov's 2017 per-event analysis (§4.3.1).

4.2 The latency budget: why handler bodies dominate

The reason the thin microbenchmark record matters less than folklore assumes is that dispatch sits at the wrong end of the latency budget. Stack the published end-to-end numbers and the picture is unambiguous.

4.2.1 Hardware and software floors

Dan Luu's 2017 logic-analyzer study measured keypress-start-to-USB-packet for 20+ keyboards: Apple Magic 15 ms; a large cluster (Logitech K120, Unicomp Model M, Filco) at 30 ms; Kinesis Advantage 50 ms; wireless Logitech K360 60 ms [83]. The spread between fastest and slowest is

~45 ms, and gaming keyboards were *not* faster [83]. His companion study put end-to-end keypress−photon at 100–200 ms on modern machines — the Apple IIe did the whole trip in 30 ms in 1983, meaning a single median modern keyboard carries more latency than an entire 1983 pipeline [84]. Fatin's 2015 decomposition (Typometer) makes the budget explicit: input averages 14 ms (debouncing 8.5 ms is the largest single term), output averages 12 ms at 60 Hz — a fixed I/O floor of ~26 ms before any application code runs [85].

Budget stage	Typical cost	Source / year
Keyboard matrix scan + debounce + USB poll/transfer	8–22 ms, avg 14 ms	Fatin 2015 [85]
Display (60 Hz refresh + pixel response)	avg 12 ms (max 17 + ~4 ms)	Fatin 2015 [85]
Fixed I/O floor	**~26 ms average** (ideal rig ~3 ms)	Fatin 2015 [85]
Whole keyboard alone (median device)	15–60 ms	Luu 2017 [83]
Application software (processing)	0.9 ms (GVim) – 49.4 ms (Atom 1.1), observed spikes to 1–2 s	Fatin 2015 [85]
End-to-end keypress−screen, modern machine	100–200 ms (2014 MBP 100 ms; 2017 Windows box 170–200 ms)	Luu 2017 [84]
Engine-side dispatch + property reads + match	nanoseconds-to-microseconds (Ch. 3; see §4.4)	this work

Fatin's editor table is the money row: on identical hardware, application-layer latency spans 0.9 ms (GVim) to 49.4 ms (Atom — a Chromium runtime) [85]. Software routinely exceeds the *entire* hardware budget. His other findings matter for keyboard work specifically: jitter is perceptually worse than constant delay; a compositing window manager imposes a mandatory ≥16.7 ms frame quantization [85]; and humans perceive latencies down to ~2 ms, so the "100 ms feels instantaneous" folk claim does not survive a terminal experiment [84].

4.2.2 Nanoseconds vs microseconds vs milliseconds

Against a 15–60 ms keyboard floor [83] and a ~26 ms I/O floor [85], a JS `keydown` handler running 5–10 ms is a significant fraction of the achievable software budget, and a >50 ms long task can dominate the whole pipeline. Dispatch itself is single-digit microseconds per event in the engine (≈4 µs for a fresh keydown at depth 1, ≈1.3 µs with a reused event object — Chapter 3). The optimization frontier is therefore not registration API choice, not capture-vs-bubble, not `key` vs `keyCode` — it is **handler-body duration and main-thread scheduling**.

4.3 Passive interventions and INP

4.3.1 The scroll interventions — and why keyboard was never in scope

The `{passive: true}` mechanism exists because of Chrome-for-Android telemetry: 80% of scroll-blocking touch listeners never call `preventDefault()`, 10% add >100 ms to scroll start,

and 1% of scrolls suffer ≥500 ms delay [86]. That motivated passive listeners (Chrome 51, 2016), then default-passive document-level `touchstart/touchmove` (Chrome 56, 2017 [87]) and `wheel` (Chrome 73, 2019 — where >98.5% of document-level wheel listeners never cancel [70]). Akulov's per-event analysis (Chrome 62 Canary, Firefox 55, 2017) tested whether `passive` changes dispatch behavior per type: wheel/touch yes in Chrome; **keydown — no benefit in either browser** [88]. Keyboard events are always main-thread, always cancelable; there is no passive fast path. For library authors: `{passive: true}` on `keydown` is a performance no-op [88].

4.3.2 INP: where handler cost becomes a ranking signal

INP (a Core Web Vital since March 2024) charges input delay + **processing (handler execution)** + presentation for clicks, taps, and key presses; a keystroke groups `keydown/keypress/keyup` and the **longest** event sets the latency [89] [90]. Thresholds at p75: good ≤200 ms, poor >500 ms [89]. A single slow `keydown` handler can therefore push a page out of "good" on its own. The field data supports the mechanism: Chrome trace analysis of Android interactions >200 ms found 18.76% blocked behind a JS-heavy (>100 ms) long task, ~10% with >100 ms of JS on the input path, 8% with a single handler >100 ms [91]. SpeedCurve RUM showed the same tail structure for FID — median 3 ms, p95 30 ms [92] — and after the FID→INP switch only ~65% of sites score "good" versus 93%+ under FID, precisely because handler processing is now billed [93]. The remedies are scheduling, not listener plumbing: `scheduler.yield()` (Chrome 129+, 2024) posts continuations as prioritized tasks so input and paint interleave [94]; Perf Planet measured chunked work at ~1 s via `scheduler.yield()` vs >2 minutes via `setTimeout(0)` (4 ms clamping), recommending ~50 ms batches and warning that `await` inside `forEach` does not yield between iterations [95].

4.4 Reconciling our Chromium-150 numbers with the literature

Chapter 3 measured, on Chromium 150: full synthetic keydown dispatch ~4.0 µs at DOM depth 1, rising to ~32 µs at depth 500; `key/code/keyCode` reads within 1–6% of each other; bitmask shortcut matching at 16.5M ops/s (~60 ns). Reconciled against §4.2's floors:

- Dispatch at realistic depth: ~4 µs is **~10^{-4} of a 30 ms keyboard trip** — four orders of magnitude down. Even the pathological depth-500 case (32 µs) is three orders below the ~26 ms I/O floor [85].
- Property reads and matching (~60 ns) sit five orders of magnitude below the floor — below timer resolution, below relevance, consistent with the literature's documented gap (§7 of the evidence record: any match-strategy difference is "undetectable against the latency budget" [78]).
- The jQuery 2.5× dispatch overhead [73], even multiplied through, lands in microseconds — real, but dwarfed the moment any handler body runs single-digit milliseconds, which Fatin's 49.4 ms Atom figure shows is the norm for Chromium-based apps [85].

The literature and our measurements agree quantitatively: engine-side event machinery is nanoseconds-to-microseconds; hardware and display are tens of milliseconds; the actionable budget is handler-body duration and main-thread scheduling — exactly the territory INP meters [89] [91].

5. Decision Matrix: Architecture by Scenario

Chapters 1–4 established the semantics, the machine, the measurements, and the budget; this chapter converts them into orders. Every prescription is justified in one breath: a measured anchor from this investigation's Chromium 150 sandbox, plus the engine reason that makes the number portable. One framing rule governs all of it: engine costs are ns–µs against a 15–60 ms hardware floor [83] and a ~26 ms I/O floor [85], so the matrix optimizes *library cleanliness and worst-case throughput* — deep DOMs, high churn, thousands of matches per interaction — while what actually hits INP's 200 ms p75 threshold [89] is your handler body. Choose architecture by scenario; spend optimization effort inside the handler.

5.1 By application type

5.1.1 Key identification: `code` for physical, `key` for semantic, `keyCode` only for legacy tables

Pick the property by *meaning*, never by speed — every accessor, legacy or modern, lands within 1–6% of the no-read baseline in the sandbox, with `e.which` literally tying it. There is no performance argument among them.

- **Games and positional layouts (WASD): use `event.code`.** `"KeyA"` is the physical position — the key "labelled q on an AZERTY keyboard" [17] — so bindings survive layout switching for free. `code` also pre-encodes `location` (`"ShiftLeft"` vs `"ShiftRight"`).
- **Shortcut managers and text UIs: use `event.key` plus the four modifier booleans.** `key` is the layout-resolved meaning — `"q"`/`"Q"`, `"Enter"` [16] — which is what "Ctrl+Q" means to a user. Add `getModifierState("Accel")` when you need the platform-correct command modifier [23].
- **`keyCode`/`which`/`charCode`: only as lookup keys into pre-existing legacy tables.** They are deprecated, "system and implementation dependent… inconsistent across platforms" [18], and report 0 on synthetic events — but the number compare is fast and old tables are keyed on them. The one modern use is defensive: `keyCode === 229` as the IME "not yours" marker [20].
- **Hoist `e.key` into a local once per handler regardless.** Blink and WebKit cache the string eagerly at construction [44] [51]; Gecko re-maps it through key-name tables *on every access* [58]. Free in two engines, a real win in the third — a zero-cost portability move.

5.1.2 Listener topology: one root listener + internal dispatch; capture poisons SkipEventCapture

For any library, the default is **one listener at `document` plus an internal dispatch table**. Position is settled: leaf-vs-root measures 1.002 at depth 1 and 1.006 at depth 500 — statistically irrelevant, because Blink builds the O(D) EventPath once and walks it regardless [47] [48]. What matters is *boundary crossings*: N listeners are N full C++→JS invocations with no batching possible (§2.4), so one fat listener switching on `e.code` in JS amortizes what many small native registrations cannot.

On phase: **register in the bubble phase unless you have a correctness reason not to.** Capture and bubble measure within 1.01 of each other — phase is free per dispatch — but any

capture listener anywhere flips Blink's document-global `HasCaptureListener()` predicate and disables the default-on SkipEventCapture optimization *for every dispatch on the page* [47] [50]; WebKit's per-document `EventListenerCounts` behaves identically [51]. A shortcut library registering `addEventListener('keydown', fn, true)` "to run first" taxes every listener-free dispatch of every type for the whole page. If you genuinely need first-mover semantics, scope the capture listener to the smallest subtree, not `window`.

Depth rarely matters because you rarely control it — but know the cliff: dispatch rises from ~4.0 µs at depth 1 to ~32 µs at depth 500 (~8×). Since path cost is paid whether or not listeners exist, the mitigation is handler-side: `stopPropagation()` on a consumed shortcut measured **17% faster** than letting it propagate. Stacking is free, spreading is not: 1/10/100 listeners on one node sit within 2.2% of each other; 10 listeners across 10 ancestors run 2.74× slower. Dispatch is path-bound, not invocation-bound.

5.2 By event volume and registration churn

5.2.1 High-churn UIs: delegation always; static apps: direct listeners fine

Delegation's dispatch-time cost is a wash — one document listener doing `e.target.closest('[data-k]')` fires at 176k ops/s against 174k for 1000 direct listeners (1.2%, inside CV). The decision is made at *registration*: one add/remove pair costs ~0.75 µs (0.744 µs precisely); 1000 direct registrations cost 766.9 µs — on the order of 1000× the delegated single registration, perfectly linear, no batching discount in Blink's listener list management. So:

- **High-churn UIs** (virtualized lists, mount/unmount cycles, per-render re-registration): delegate, unconditionally. Delegated registration is O(1) in control count; direct is O(N) at ~0.77 µs per control, paid again on every teardown.
- **Static apps** with a bounded handful of controls: direct listeners are fine — registration is paid once, dispatch is at parity either way.
- The delegated selection predicate is a correctness choice, not a speed one: `closest`, `matches`, `tagName`, and `composedPath()[0]` all measured within 4.2%. Use `closest` for clarity; avoid `composedPath()` in loops only because it allocates a fresh array per call [10].

5.2.2 Teardown: `removeEventListener` for hot paths; `AbortSignal` for ergonomic bulk teardown

Removing 100 listeners explicitly costs 44.2 µs per full cycle; the AbortController route — 100 signal-bound registrations plus one `abort()` — costs 83.8 µs, **1.9× slower end-to-end**, because signal bookkeeping at registration outweighs single-call removal, and `abort()` still runs K individual removal algorithms with no batch fast path (§2.4). Verdict: `AbortSignal` is an ergonomics and leak-safety feature — one `abort()` retires an entire modal scope without storing callback references [1] — not a speed feature. Use it for scope teardown at human timescales (dialog close, route change); use explicit `removeEventListener` where teardown itself is hot. `once: true` sits between them: the full register+dispatch+auto-remove cycle runs only ~34% above the plain dispatch baseline — fine for fire-once shortcuts. Whichever you

choose, keep stable function references: anonymous callbacks defeat both dedup and removal, since matching keys on (type, callback, capture) only [6].

5.3 The master decision table

Scenario → listener topology → key-identification strategy → matching structure → measured anchor → engine reason. All measurements from this investigation's Chromium 150 sandbox; spec-level claims cited to source. For the library-source-level internals behind rows 1–3 — how shipping shortcut engines implement the trie, the chord state machine, the scope stack — see Chapter 6.

#	Scenario	Listener topology	Key ID strategy	Matching structure	Measured anchor	Engine reason
1	Global shortcut library	One document bubble listener + internal dispatch	e.key + modifier booleans	Bitmask int → Map<number>	16.5M ops/s vs 5.4M string (3.1×)	N listeners = N C++→JS crossings; string concat, not the hash, is the cost
2	Multi-stroke chords	Same root listener, JS state machine	e.code for position, e.key for mnemonics	Map-of-Maps trie	14.6M ops/s (0.88 of bitmask; tail-latency caveat)	Same amortization; trie depth costs one Map.get per stroke
3	Command palette / per-keystroke filtering	Root listener + input observer	e.key	Bitmask + Map, precomputed	Scan of 50 bindings: 2.3M ops/s, 7.2× slower	Thousands of matches per interaction is the only regime where matching shows
4	Text editor / contenteditable	Direct listener on editable root; document fallback	e.key; guard isComposing / keyCode===229	Early-exit guards, then Map	Guards <0.5% (under noise floor)	Keyboard default actions run synchronously after dispatch — a slow handler delays insertion directly
5	Game loop (WASD, held keys)	window keydown+keyup pair, bubble; key-state Set	e.code (layout-independent [17])	Map<code→action> or bitmask; first-line e.repeat guard	Property reads within 1–6%; repeat guard <0.5%	Each auto-repeat tick is one full pipeline traversal — 30 dispatches/s, no engine throttling
6	Form UI / dialogs / focus traps	Delegated document listener; cancel keydown for Tab containment [29]	e.key ("Tab", "Escape", "Enter")	closest() scope check + Map	Delegation dispatch parity (176k vs 174k)	Shortcut scope is focus scope; keydown cancellation blocks focus movement [13]
7	Global hotkeys ("run first")	Smallest-subtree listener, bubble; capture only with justification	e.key + getModifierState ("Accel") [23]	Bitmask + Map	capture≈bubble 1.01 at depth 500	Any capture listener disables SkipEventCapture page-wide [47] [50]

#	Scenario	Listener topology	Key ID strategy	Matching structure	Measured anchor	Engine reason
8	High-churn UI (mount/unmount per render)	Delegation, always	Either	Either	Registration 0.74 µs/pair → 1000× for 1000 listeners	No batching discount; direct is O(N) at setup *and* teardown
9	Static app, few controls	Direct listeners on the controls	Either	Either	1/10/100 same-node listeners within 2.2%	Path computed once; per-node invocation cheap; registration paid once
10	Deep DOM (>100 depth)	Single root listener (position irrelevant)	Either	Either	4.0 µs @depth1 → 32 µs @depth500 (~8×)	O(D) EventPath built per dispatch, never cached; paid even with zero listeners
11	Anti-pattern: handlers spread up the tree	Consolidate onto one node	—	—	10 listeners / 10 ancestors: 3× slower	Each ancestor with listeners pays snapshot clone + boundary crossing
12	Modal/route scope teardown	Any + AbortController({signal})	—	—	abort() route 1.9× slower than explicit removal for 100	Per-listener abort algorithms, no batch fast path — pay for ergonomics knowingly
13	Teardown on a hot path	Explicit removeEventListener, stable references	—	—	44 µs vs 84 µs per 100-listener cycle	Plain map scan + vector erase; matching keys on (type, callback, capture) only [6]
14	Fire-once shortcut	Any + once: true	Either	Either	183k ops/s full cycle, +34% over baseline	Auto-removal at invoke time is modest bookkeeping [1]
15	Consuming a handled shortcut	Any — call stopPropagation() + preventDefault()	—	—	stopPropagation 17% faster than propagating; preventDefault within noise	Skips remaining path walk; cancel bit suppresses beforeinput/input (200−0 trusted events) [13]
16	IME-heavy input (CJK)	Any; guards non-negotiable	Skip while e.isComposing; treat 229 as “not yours” [20]	Guard first, match second	Guards <0.5%	keydown fires throughout composition with isComposing: true [13]; divergence is

#	Scenario	Listener topology	Key ID strategy	Matching structure	Measured anchor	Engine reason
						per-browser
17	Synthetic event bus / test tooling	Any — reuse one preconstructed event	Constructor sets key/code; keyCode stays 0 [26]	Either	Reuse: 776k vs 246k ops/s — 3.2×	Construction is over half of construct+dispatch; only synthetic flows may pool events
18	Legacy keymap integration	Any	keyCode/which as legacy table keys only	Number-keyed Map	e.which ties no-read baseline (1.000)	Deprecated and platform-inconsistent [18] — but the number read itself is free

Two patterns deserve code, because they are the ones libraries get wrong. The canonical hot handler — guard, hoist, mask, map:

```
const bindings = new Map(); // bitmask int -> command
function mask(e) { // 4 modifier bits + key index: one integer, no strings
  return (e.shiftKey | e.ctrlKey << 1 | e.altKey << 2 | e.metaKey << 3) << 8 | keyIndex(e);
}
addEventListener('keydown', e => {            // bubble: keep SkipEventCapture alive
  if (e.repeat || e.isComposing || e.keyCode === 229) return; // free guards
  const cmd = bindings.get(mask(e));          // 16.5M ops/s vs 5.4M for string keys
  if (!cmd) return;
  e.preventDefault(); e.stopPropagation();    // consumption is an optimization, not overhead
  cmd(e);
});
```

And the honest teardown split — ergonomics at scope boundaries, explicit removal where teardown is hot:

```
// Scope teardown (dialog close): pay the 1.9x for leak-safety.
const scope = new AbortController();
el.addEventListener('keydown', onKey, { signal: scope.signal });
closeButton.onclick = () => scope.abort();   // retires every listener in the scope

// Hot-path churn (per-render): explicit removal wins 44 µs vs 84 µs per 100.
function unmount() { el.removeEventListener('keydown', onKey); } // stable reference required
```

Everything the matrix omits is deliberate. `passive` on `keydown` is a performance no-op — keyboard was never in the intervention's scope [88]. `on*` property handlers and `{handleEvent}` objects cost 8–10% over a plain function — acceptable for stateful or legacy-interop listeners, wrong for the hot global keymap. And no row moves the INP needle by itself: dispatch at realistic depth is ~10^{-4} of a 30 ms keyboard trip. The matrix buys a clean, worst-case-safe architecture in microseconds; the remaining milliseconds — handler-body duration, long tasks, scheduling with `scheduler.yield()` [94] — are yours to spend or save.

6. The Library-Author Playbook

Chapters 3–5 measured what engines charge; this chapter reverse-engineers what the fastest shipping shortcut engines actually build. The surveyed codebases — VS Code, CodeMirror 6, ProseMirror, Mousetrap, hotkeys-js, tinykeys, kbar, xterm.js, react-hotkeys-hook, and React itself — converge on a small set of architecture patterns, and the sandbox numbers from Chapter 3 explain why. Where they diverge, the divergence is measurable.

Table 6-1. Per-library architecture at a glance (source-level facts; citations per row)

Library	Attachment	Key ID	Encoding	Matching	Chords
VS Code workbench	One root keydown → KeybindingService._dispatch [96]	KeyCode enum via quirk-table normalization; scan-code dispatch by default [97] [98]	Bitmask integer (_computeKeybinding) [97]	_map: Map<firstChord, items[]> + backward when scan [99]	Multi-chord, 5 s timeout, IME disabled in chord mode [96]
CodeMirror 6	One keydown on contentDOM via domEventHandlers [100] [101]	w3c-keyname keyName(event) + keyCode fallback [100]	Normalized string, Alt-Ctrl-Meta-Shift-order [100]	Plain-object map, **WeakMap-cached** per facet value [100]	Multi-stroke, 4000 ms prefix timeout [100]
ProseMirror	Plugin prop handleKeyDown on view DOM [102]	w3c-keyname + guarded keyCode fallback [102]	Same normalized-string convention [102]	Plain-object map, direct lookup [102]	None
xterm.js	keydown on hidden textarea [103]	Legacy keyCode switch + surgical key/code fallbacks [103]	4-bit modifier mask for CSI sequences [103]	Jump-table switch [103]	None
hotkeys-js	keydown+keyup per element	Numeric keyCode; layout-	Numeric keyCodes; modifiers as	keyCode bucket +	None (simultaneous

Library	Attachment	Key ID	Encoding	Matching	Chords
	(elementEventMap) [104] [105]	independent hybrid since v4.3 [106]	[91,17,16] [107]	array scan + compareArray [108]	only) [104]
Mousetrap	3 listeners (keypress/keydown /keyup) per instance [109]	e.which/ e.keyCode → char; US-assumption _SHIFT_MAP [109]	Char-keyed object _callbacks [109]	Bucket + linear scan, sort().join(',') modifier compare [109]	Sequences, 1000 ms reset [109]
tinykeys	Exactly one listener per target [110]	event.key **and** event.code; getModifierState [110]	Parsed AST: KeybindingPress tuple [110]	Linear scan of all parsed bindings [110]	pending map, 1000 ms timeout [110]
kbar	Vendors tinykeys; two window-level listeners total [111] [112]	Inherits tinykeys [111]	Space-joined sequences [111]	Inherits tinykeys scan; actions pre-sorted by length [111]	Yes, 400 ms timeout [111]
react-hotkeys-hook	One listener pair **per hook call** [113]	event.code default since v5; useKey opt-out [114]	Parsed Hotkey object [113]	forEach over parsed keys [113]	possibleMatches map, 1000 ms [113]
React 17+	Root-container delegation for all event types [115] [116]	getEventKey legacy normalization; code since #18287 [117] [116]	n/a (synthetic events)	Fiber-tree walk per dispatch [118]	n/a

6.1 Attach once, dispatch internally

6.1.1 The single-listener registry

The dominant pattern is one native listener plus a library-owned registry. VS Code attaches a single `keydown` at the workbench root and routes every keystroke through `KeybindingService._dispatch` [96]. tinykeys attaches exactly one listener on the caller's target [110]; kbar, which vendors tinykeys, runs an entire command palette — toggle plus every action shortcut — on two window-level listeners [111] [112]. React generalizes the same idea to all event types: PR #18195 moved delegation from `document` to the root container, and PR #19659 attaches all known listeners when the root mounts, so zero application listeners ever touch native DOM nodes directly [115] [116].

```
// Delegation root listener — the only native registration a shortcut
engine needs
const registry = new Map(); // chordKey -> handler
root.addEventListener("keydown", (e) => {
  if (e.isComposing || e.repeat) return;       // guards cost <0.5%
(Ch 3, Table 3-8)
  const h = registry.get(encode(e));           // one Map lookup, no
re-parse
  if (h && h(e) !== false) { e.preventDefault();
e.stopPropagation(); }
});
```

The economics are measured, not aesthetic. In this investigation's sandbox (Chromium 150), one root listener doing internal matching dispatched at 176.1k ops/s against 174.0k for 1000 direct per-target listeners — parity within CV — while registration cost 0.75 µs for the delegated architecture versus 766.9 µs for the direct one, a ~1000× setup gap that is perfectly linear in listener count (Tables 3-4, 3-5). Per-listener registration is O(N); the registry is O(1). A library that mounts N hotkey-bearing components therefore has exactly one viable topology.

Delegation also buys control the native dispatch order cannot: the library decides precedence (VS Code's backward scan lets user rules shadow defaults [99]; CM6's facet order resolves competing keymaps [100]), and it decides teardown granularity. Note, though, that bulk teardown via `AbortController.abort()` measured 1.9× slower than explicit `removeEventListener` loops in the sandbox (83.8 µs vs 44.2 µs per 100-listener cycle, Table 3-13) — engine removal scans per aborted registration — so with a one-listener registry the question is usually moot: you remove one listener and drop the Map.

6.1.2 Outlier anti-pattern: react-hotkeys-hook per-hook listeners

react-hotkeys-hook attaches `keydown/keyup` on `ref.current || options.document || document` inside each `useHotkeys` call — N native listener pairs for N hotkeys, no central registry [113]. The API ergonomics are real, but the architecture pays O(N) registration and, worse, invites churn: hooks that re-bind on render multiply the 0.74 µs-per-pair registration tax (Table 3-5 shows add and remove are symmetric). The same library's own evolution — v5 switching to `event.code` matching "to get rid of all the confusion between different keyboard layouts, multiple accidental triggers" [114] — shows its maintainers optimizing the match layer while the attachment layer remains the outlier. The fix, demonstrated by kbar over tinykeys, is a module-level manager that registers hooks into one map and owns the single listener [111].

6.2 Parse at registration, match at dispatch

6.2.1 Three encoding families

Every serious library converts shortcut strings to a dispatch-time-efficient form once, at `bind()` time. The surveyed codebases cluster into three families.

Bitmask integers. VS Code's `_computeKeybinding()` ORs `KeyMod.CtrlCmd | KeyMod.Alt | KeyMod.Shift | KeyMod.WinCtrl | keyCode` into one integer, making `equals()` a single `===` [97]. xterm.js uses the canonical 4-bit variant `(shift?1:0)|(alt?2:0)|(ctrl?4:0)|(meta?8:0)` — which doubles as the CSI modifier protocol parameter [103].

```
// Bitmask encode + Map<number>: 16.5M ops/s in the sandbox (Table 3-7)
const CTRL = 1 << 28, ALT = 1 << 29, SHIFT = 1 << 30, META = 1 << 27;
function encode(e) {
  return (e.ctrlKey ? CTRL : 0) | (e.altKey ? ALT : 0)
       | (e.shiftKey ? SHIFT : 0) | (e.metaKey ? META : 0) |
e.keyCode;
}
registry.set(CTRL | 75, handler); // dispatch: registry.get(encode(e))
```

Normalized strings. CM6 and ProseMirror build `"Alt-Ctrl-Meta-Shift-<key>"` names with canonical modifier order, platform-resolved `Mod-`, and `Space→" "` [100 102]:

```
// Normalized-string builder — readability-first; 3.1× slower than bitmask (Table 3-7)
function keyName(e) {
  let n = "";
  if (e.altKey) n += "Alt-"; if (e.ctrlKey) n += "Ctrl-";
  if (e.metaKey) n += "Meta-"; if (e.shiftKey) n += "Shift-";
  return n + (e.key === " " ? "Space" : e.key); // then map[n] — string build is the cost
}
```

Registration-time ASTs. tinykeys parses each binding once into `KeybindingPress = [requiredModifiers[], optionalModifiers[], key | RegExp]` — no string is ever built at dispatch; matching is field comparison plus `getModifierState` [110]:

```
// Registration-time parse: dispatch never touches the raw binding string
function parse(binding) { // "$mod+Shift+K" -> [["Meta","Shift"], [], "K"]
  const parts = binding.split(/\+/), key = parts.pop();
  return [parts.map(p => p === "$mod" ? (isMac ? "Meta" : "Control") : p), [], key];
}
```

Table 6-2. Encoding families compared (ops/s from this investigation's sandbox, Chromium 150, handler-only, Table 3-7)

Family	Exemplars	Dispatch op	Measured	Trade
Bitmask int + Map<number>	VS Code [97], xterm [103]	int compare	16.5 M ops/s (60 ns)	Fastest; exact-equality by construction; opaque to debug
Map-of-Maps trie	(sequence engines)	chained gets	14.6 M ops/s; tail latency (CV 18.6%)	Natural for chords; deopt-prone
Normalized string + Map.get	CM6, ProseMirror [100] [102]	string build + hash	5.4 M ops/s (3.1× slower than bitmask)	Human-readable bindings; allocation per keystroke
AST scan (no string built)	tinykeys [110]	field compares × N	~2.3 M ops/s at N=50 (7.2× slower)	Zero-grammar; fine while N is small

The sandbox validates VS Code's choice quantitatively: string construction, not the hash lookup, is what costs 3.1×. But the proportionality check matters — inside a real 4 µs dispatch even the 438 ns scan is swamped, so these families differentiate only at thousands of matches per interaction (chord sequencers, per-keystroke palette filtering).

6.2.2 Matching structures

The pattern is *index by the cheapest discriminant, then scan a tiny candidate set*. VS Code buckets every keybinding by its first chord into `_map: Map<firstChord, ResolvedKeybindingItem[]>`, so per-keystroke work is one Map get plus a backward scan evaluating `when` expressions — and even that scan was optimized after issue #129625 showed a complex `when` clause adding ~10 s to startup; the resolver now substitutes constants before bucketing (#174218) [99] [119]. hotkeys-js buckets by keyCode in `_handlers` and scans the bucket array with `compareArray` for modifiers [108]. Mousetrap buckets by character in `_callbacks` and compares sorted-joined modifier strings [109]. CM6 goes furthest on caching: `buildKeymap`'s flattened scope map is stored in a `WeakMap` keyed by the facet's value array, so facet recomputation — not per-keystroke work — pays for the build [100]. tinykeys is the deliberate exception: a full linear scan per keystroke, viable only because typical N is in the tens — the measured 2.3M ops/s floor for a 50-binding scan is exactly the ceiling that decision accepts [110].

6.3 Chords, IME, and the traps

6.3.1 Timeout-based chords

Every chord engine is a state machine plus a timer; the disagreements are only in timeout and prefix handling. tinykeys and Mousetrap reset pending sequences after 1000 ms [110] [109]; kbar tightens to 400 ms [111]; CM6's `PrefixTimeout` is 4000 ms [100]; VS Code allows 5 s of idle on a 500 ms `IntervalTimer` tick and additionally exits chord mode on focus loss [96]. Two subtleties separate the careful implementations. First, **prefix keystrokes must be swallowed**: CM6 registers synthetic prefix bindings that always `preventDefault`, and VS Code's

`MoreChordsNeeded` resolution sets `shouldPreventDefault = true` — otherwise `ctrl+k z` types `z` when the chord dead-ends [100] [96]. Second, **ambiguity must not double-fire**: tinykeys warns and refuses rather than firing both a short binding and a pending longer one [110]; kbar pre-sorts actions by descending shortcut length and wraps handlers in a `WeakSet` dedupe (its workaround for tinykeys issue #37) [111]. VS Code additionally **disables the IME on entering chord mode** and re-enables it on exit, so a composing IME cannot swallow the second chord [96].

6.3.2 IME guards and platform traps

IME handling is where shortcut engines earn correctness. VS Code normalizes *every* composing keystroke — not just `keyCode === 229` — to the sentinel `KeyCode.KEY_IN_COMPOSITION`, because "some platform/IME combinations report the real key code for keys the IME owns — notably the Enter that commits a composition, but also Space, Escape and the arrows"; all three dispatch entry points bail on it [97] [96]. tinykeys' default filter ignores `event.isComposing` and `event.repeat` [110]. xterm.js routes composition through a separate `CompositionHelper` and never converts composing keydowns to escape sequences [103].

```
// IME guard — hoist reads once per handler; guards measure <0.5%
(Table 3-8)
function onKeydown(e) {
  const { key, code, isComposing, repeat } = e; // Blink caches
eagerly, but Gecko
  if (isComposing || key === "Process") return;  // re-maps per access
— read once
  if (repeat) return;
  if (isAltGraph(e)) return; // Windows: ctrlKey && altKey === AltGr
(CM6/ProseMirror guard)
  // ...match
}
```

The platform traps are codified in source, and a library that doesn't copy them ships their bugs: **AltGraph on Windows reports ctrl+alt**, so CM6/ProseMirror skip the keyCode fallback when `windows && ctrlKey && altKey` (ProseMirror issues #668/#1060/#1529), tinykeys exempts Control+Alt from its unexpected-modifier check, and VS Code ships no default `Ctrl+Alt+[char]` bindings on Windows at all [100] [102] [110] [98]. **macOS Option** produces dead keys and alternate characters; xterm.js detects `key === 'Dead'` and recovers the letter from `event.code` (issue #3725) [103]. Strict modifier equality (tinykeys fails the match on any extra held modifier [110]) prevents `Ctrl+D` firing on `Ctrl+Shift+D`. One engine caveat from the sandbox: `e.key`/`e.code` reads are within 1–6% of baseline in Blink (eager-cached getters, Table 3-6), but Gecko re-maps per access — hoist the reads once per handler as above, which also matches what `StandardKeyboardEvent`'s normalization constructor does architecturally [97].

6.4 preventDefault discipline and the 12-point playbook

Claiming the event is the last hot-path decision, and the libraries split into three disciplines: return-value conventions (Mousetrap/hotkeys-js: callback returning `false` ⇒ preventDefault + stopPropagation [109] [108]; ProseMirror/CM6: first handler returning `true` claims the event [102] [101]), resolver-driven claiming (VS Code sets `shouldPreventDefault` on match, on pending chords, and on chord dead-ends [96]), and explicit per-binding flags (CM6's `preventDefault: true` claims the key even when the command returns false [101]). The sandbox adds two non-obvious facts: `preventDefault()` itself is within noise on synthetic dispatch (Table 3-10), but on the trusted path it fully suppresses `beforeinput/input` and *raises* throughput to 635 presses/s (Table 3-14) — claiming early is cheap and downstream work disappears. `stopPropagation()` measured 17% faster than letting the event propagate, because it skips the remaining path walk (Table 3-10): precise suppression is a genuine optimization, not just hygiene. And since handler bodies dominate the ~26 ms I/O floor and INP budget that Chapter 5 budgets, the hot path stays side-effect-free — VS Code skips telemetry for `cursor|delete|undo|redo|tab|clipboard` commands for exactly this reason [96].

The 12-point playbook:

1. **Attach once, dispatch internally.** Root listener + registry at dispatch parity with 1000 direct listeners (176k vs 174k ops/s), ~1000× cheaper registration (0.74 µs/pair) [96] [110].
2. **Never per-component listeners.** Per-hook attachment (react-hotkeys-hook) is O(N) registration with symmetric churn cost [113].
3. **Parse at registration, match at dispatch.** CM6 even WeakMap-caches the built map against the facet value; per-keystroke re-parsing is a design bug [100] [110].
4. **Bitmask + `Map<number>` when the hot path matters.** 16.5M ops/s, 60 ns/lookup — VS Code's choice, quantitatively validated; strings are 3.1× slower [97].
5. **Normalized strings when readability matters.** Canonical `Alt-Ctrl-Meta-Shift-` order, platform `Mod-`; accept the 5.4M ops/s ceiling [100] [102].
6. **Linear scans only below small N.** 50-binding scan = 2.3M ops/s, 7.2× off bitmask; tinykeys accepts this by design [110].
7. **Choose `code` vs `key` deliberately and document it.** `code` for physical bindings (react-hotkeys-hook v5 [114], games), `key` for produced characters (tinykeys [110]), hybrid fallback (hotkeys-js [106]).
8. **Strict modifier equality.** Extra held modifiers must fail the match (tinykeys [110]); bitmask encode gives exactness by construction.
9. **Guard IME and repeat always.** `isComposing/repeat` guards measure <0.5% (Table 3-8) — no excuse; chord mode additionally disables the IME (VS Code [96]).
10. **Codify AltGraph/Option/quirk tables.** Windows AltGraph = ctrl+alt guards [100] [102]; macOS dead-key recovery via `code` [103]; one browser quirk table, not scattered fixes [97].
11. **preventDefault precisely, never blanket.** Resolver/per-binding flags [96] [101]; `stopPropagation` on claim is 17% *faster* than propagating (Table 3-10); check `event.defaultPrevented` at global toggles (kbar [111]).

12. **Keep the handler body side-effect-free.** Hoist `e.key/e.code` reads once (Gecko remaps per access); defer telemetry/logging (VS Code `HIGH_FREQ_COMMANDS` [96]); handler bodies, not dispatch, dominate the INP budget.

7. Appendix: Methodology, Sandbox, Caveats

All numbers in Chapters 3–4 come from a single sandbox: headless **Chrome/150.0.7871.181** (real Blink + V8), driven by Node v20.20.2 via puppeteer-core, in a Linux container. The full harness ships with this chapter; everything below is reproducible from the file inventory in §7.4.

7.1 Two measurement layers

The sandbox measures two deliberately separated layers. Conflating them is the most common category error in published event benchmarks [79] [120].

Layer A — synthetic `dispatchEvent()` (in-page). Each case loop dispatches a constructed `KeyboardEvent` on the Blink main thread:

```
// fixtures: KINIT = {key:'k', code:'KeyK', keyCode:75, which:75,
bubbles:true, cancelable:true}
t.leaf.addEventListener('keydown', H);
fn(){ t.leaf.dispatchEvent(kev()); }   // timed op: construct +
EventPath + invoke
```

`dispatchEvent` invokes handlers **synchronously** on a nested call stack; real input is queued on the event loop by the browser [79]. So Layer A measures pure in-page cost — event construction, EventPath computation, capture/bubble traversal, listener invocation — with no IPC, no input pipeline, no compositor, and no task-scheduling contamination [79] [80]. What it *can* prove: relative costs of path depth, listener count/placement, phases, registration, property reads, matching strategies. What it *cannot* prove: anything about real keypress latency, default actions, or the input pipeline. Synthetic keydown quirks matter here: `isTrusted === false`, `keyCode/which` are **0 unless explicitly set** in the init dict (we set them), and no default actions fire — synthetic `beforeinput/input` on a detached, non-editable target triggers no editing behavior.

Layer B — trusted input (`page.keyboard.press` → CDP `Input.dispatchKeyEvent`). The Node driver sends 200 keydown→keyup presses of `'k'` into a focused `<input>`, wall-clock timed driver-side; a second pass (50 presses) sets an in-page `performance.now()` marker immediately before each dispatch and the capture-phase keydown handler computes the delta. Trusted events traverse Chromium's real input pipeline and are scheduled through the input event queue, so per-press times are task/frame-quantized and far noisier than Layer A [79]. Measured throughput — **234–252 presses/s** — is **driver/pipeline-bound** (CDP round-trip Node→Chromium plus pipeline scheduling), **not engine-bound**: it cannot saturate the JS dispatch path, which Layer A shows runs at ~250k ops/s. The 1.3–1.4 ms avg latency figures include CDP marker round-trip overhead; treat them as upper bounds on true input-to-handler latency. Layer B exists to verify behavioral facts (keydown→beforeinput→input ordering;

`preventDefault()` on keydown fully suppresses both and raises throughput to 635 presses/s by skipping character insertion), not to benchmark handler speed.

7.2 Harness mechanics and fixtures

Timing follows the fixed-window style of jsben.ch-class harnesses [120], with explicit calibration. Per case: ~200 ms warmup to tier the JIT past baseline, then 5 measured reps of ≥300 ms each; batch size doubles until a batch takes ≥2 ms (keeping timer overhead and `performance.now()` granularity — reduced post-Spectre [81] — negligible against batch duration):

```
function measureOnce(fn, ms){
  let batch = 1;
  for(;;){ const t0 = now(); for(let i=0;i<batch;i++) fn(i);
    if(now()-t0 >= 2 || batch >= (1<<26)) break; batch *= 2; }
  let ops = 0; const tStart = now();
  do { for(let i=0;i<batch;i++) fn(i); ops += batch; } while(now() -
tStart < ms);
  return ops / ((now() - tStart)/1000);
}
```

Anti-bias controls, all in `bench.html::runSuites`: **alternating case order** across reps (rep 0 forward, rep 1 reversed, …) to cancel ordering/drift effects; **`gc()` between suites** via `--js-flags=--expose-gc`; **`setTimeout(0)` yields** between suites and reps so the event loop breathes; and a **DCE sink guard** — every handler accumulates into `window.__sink` (`const H = () => { sink(1); }`), and the driver verifies the final sink is nonzero, so no handler body can be dead-code-eliminated. Reported values are medians of pooled reps, never means of single runs.

Launch configuration (driver, `run.js`):

```
puppeteer.launch({ executablePath: '/usr/bin/chromium', headless:
true,
  args: ['--no-sandbox','--disable-gpu','--disable-dev-shm-usage','--
js-flags=--expose-gc'] });
```

Fixtures: **detached** depth trees of 1/10/50/500 nested `<div>`s (detached so path length == tree depth exactly; attached trees add 2–3 constant body/html/document entries — scaling is identical, confirmed by the attached flat-tree cases); an **attached container of 1000 `<button data-k>`**; an **attached 2000-node flat sibling tree**; and a focused `<input>` for Layer B. The linear-scan matching case places the hit **last** of 50 bindings (worst case; average ≈2× faster).

7.3 Stability and honesty flags

Two full runs were executed; per-case medians agreed **within ±10% for 61 of 63 cases**. The 2 outliers were rerun once and pooled (15 reps instead of 10). One case — `capture-vs-bubble :: bubble@leaf` — showed a genuinely **bimodal distribution** (~26k vs ~32k ops/s across

runs), a JIT/GC tier shift; it is reported at its pooled median (30.9k) and flagged, not smoothed away. The Map-of-Maps trie case had one deopt rep (CV 18.6%). Bimodality of this kind is inherent to micro-benchmarking tiered JITs: tight loops amortize tier-up that real ~150 ms-cadence keypress handlers never get [85] [120], so Layer A numbers describe steady-state dispatch, not cold-press reality. All **absolute numbers are machine-specific to this container**; single engine (Blink/V8), headless only. The portable results are the *ratios*.

7.4 Reproduce and extend

File	Role
key-bench/bench.html	In-page harness (runSuites, measureOnce, stats), all 14 suites, fixtures, Layer B counters
key-bench/run.js	puppeteer-core driver: launch, 2 full runs, outlier rerun, Layer B loop, writes all JSON
key-bench/gen_report.js	Renders RESULTS.md tables from results.json
key-bench/ results_run1.json	Raw run 1 (per-rep samples retained)
key-bench/ results_run2.json	Raw run 2
key-bench/ results_rerun.json	Rerun reps for the 2 outlier cases
key-bench/results.json	Merged medians/CV/agreement flags + Layer B data
key-bench/RESULTS.md	Rendered tables, findings, caveats

Rerun end-to-end:

```
cd key-bench && npm i puppeteer-core && node run.js && node
gen_report.js
```

To extend: add a `suite(name, cases)` block to `bench.html`; each case is `{name, fn, setup?, before?, cleanup?}` where `fn` is the timed op. Keep handler results flowing into `sink()`, keep fixtures detached unless attachment is the variable, and let the outlier-rerun logic in `run.js` arbitrate disagreement. To port to another engine, point `executablePath` at its binary; expect absolutes to move and ratios to hold.

References

1 TypeScript `lib.dom.d.ts` / DOM spec text of addEventListener ("same type, callback, and capture"; capture/passive/once/signal semantics) — https://github.com/Steve-xmh/applemusic-like-lyrics/blob/main/packages/core/docs/classes/LyricPlayer.md (accessed 2026-08-14). https://github.com/Steve-xmh/applemusic-like-lyrics/blob/main/packages/core/docs/classes/LyricPlayer.md

2 MDN — `EventTarget.addEventListener()` (incl. options semantics, passive defaults, dedup warning) — https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener ; https://github.com/mdn/content/blob/main/files/en-us/web/api/eventtarget/addeventlistener/index.md (accessed 2026-08-14). https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener

3 W3C DOM 4.1 WD — Event flags, stopPropagation/stopImmediatePropagation/preventDefault algorithms — https://www.w3.org/TR/2017/WD-dom41-20171207/ (accessed 2026-08-14). https://www.w3.org/TR/2017/WD-dom41-20171207/

4 WHATWG HTML Standard — §8.1.8 Events / event handlers (IDL attributes act as non-capture listeners; ordering example; GlobalEventHandlers IDL) — https://html.spec.whatwg.org/multipage/webappapis.html (accessed 2026-08-14). https://html.spec.whatwg.org/multipage/webappapis.html

5 Stefan Judis — "addEventListener accepts functions and objects" (handleEvent object listeners; `this` semantics; not usable via on*) — https://www.stefanjudis.com/today-i-learned/addeventlistener-accepts-functions-and-objects/ (accessed 2026-08-14). https://www.stefanjudis.com/today-i-learned/addeventlistener-accepts-functions-and-objects/

6 MDN (mirror) — `EventTarget.removeEventListener()` matching rules ("only the capture setting matters") — https://udn.realityripple.com/docs/Web/API/EventTarget/removeEventListener (accessed 2026-08-14). https://udn.realityripple.com/docs/Web/API/EventTarget/removeEventListener

7 W3C DOM Core WD — invoke event listeners algorithm (stop immediate propagation termination; capture filtering) — https://www.w3.org/TR/2011/WD-domcore-20110531/ (accessed 2026-08-14). https://www.w3.org/TR/2011/WD-domcore-20110531/

8 W3C DOM 4.1 WD (2017-10) — §3.8 Dispatching events algorithm — https://www.w3.org/TR/2017/WD-dom41-20171021/ (accessed 2026-08-14). https://www.w3.org/TR/2017/WD-dom41-20171021/

9 W3C DOM Level 3 Events — propagation path is fixed once determined (quoted) — https://www.w3.org/TR/DOM-Level-3-Events/ ; https://www.cnblogs.com/Ox9A82/p/6227765.html (accessed 2026-08-14). https://www.w3.org/TR/DOM-Level-3-Events/

10 Polymer — Shadow DOM concepts: event retargeting & composedPath — https://polymer-library.polymer-project.org/2.0/docs/devguide/shadow-dom (accessed 2026-08-14). https://polymer-library.polymer-project.org/2.0/docs/devguide/shadow-dom

11 javascript.info — "Shadow DOM and events" (composedPath contents, open/closed roots, composed true/false event lists) — https://javascript.info/shadow-dom-events (accessed 2026-08-14). https://javascript.info/shadow-dom-events

12 greadme — "What Are Passive Event Listeners?" (Chrome document-level passive defaults table) — https://www.greadme.com/blog/best-practices/improve-scrolling-with-passive-event-

listeners-complete-guide (accessed 2026-08-14). https://www.greadme.com/blog/best-practices/improve-scrolling-with-passive-event-listeners-complete-guide

13 W3C UI Events — KeyboardEvent event types (keydown/keyup/keypress tables), §3.6.5/3.6.6 composition ordering, §4.3.4 cancelable keydown quote, repeat quote, legacy key attributes — https://www.w3.org/TR/uievents/ ; https://w3c.github.io/uievents/ (accessed 2026-08-14). https://www.w3.org/TR/uievents/

14 W3C Input Events Level 2 — beforeinput/input, cancelability requirements, IME inputTypes — https://www.w3.org/TR/input-events-2/ (accessed 2026-08-14). https://www.w3.org/TR/input-events-2/

15 whatwg/webcomponents issue #513 — list of events with `composed: true` — https://github.com/w3c/webcomponents/issues/513 (accessed 2026-08-14). https://github.com/w3c/webcomponents/issues/513

16 W3C UI Events KeyboardEvent key Values — key selection algorithm; `"Dead"`, `"Process"`, `"Unidentified"`; IME/composition key table — https://www.w3.org/TR/uievents-key/ (accessed 2026-08-14). https://www.w3.org/TR/uievents-key/

17 W3C UI Events KeyboardEvent code Values — code tables (KeyA "Labelled q on an AZERTY keyboard", IntlBackslash, etc.) — https://www.w3.org/TR/uievents-code/ (accessed 2026-08-14). https://www.w3.org/TR/uievents-code/

18 MDN — `KeyboardEvent.keyCode` (deprecated; "system and implementation dependent numerical code…") — https://developer.mozilla.org/en-US/docs/Web/API/KeyboardEvent/keyCode (accessed 2026-08-14). https://developer.mozilla.org/en-US/docs/Web/API/KeyboardEvent/keyCode

19 inexorabletash — KeyboardEvent polyfill docs (legacy keyCode/charCode/which semantics; IE/Windows VK-code heritage) — http://inexorabletash.github.io/polyfill/keyboard.html (accessed 2026-08-14). http://inexorabletash.github.io/polyfill/keyboard.html

20 javascript.info — "Keyboard: keydown and keyup" (auto-repeat; mobile keyboards keyCode 229 / key "Unidentified") — https://javascript.info/keyboard-events (accessed 2026-08-14). https://javascript.info/keyboard-events

21 TypeScript KeyboardEvent interface listing (DOM_KEY_LOCATION_* constants, location, isComposing, getModifierState) — https://docs.chartbreaker.com/interfaces/bundle._internal_.KeyboardEvent.html (accessed 2026-08-14). https://docs.chartbreaker.com/interfaces/bundle._internal_.KeyboardEvent.html

22 Mozilla bug 1594003 — `KeyboardEvent.repeat` always false on X11/XINPUT2 — https://bugzilla.mozilla.org/show_bug.cgi?id=1594003 (accessed 2026-08-14). https://bugzilla.mozilla.org/show_bug.cgi?id=1594003

23 MDN (mirror) — `KeyboardEvent.getModifierState()` modifier tables per platform (AltGraph, CapsLock, OS…) — https://web.nodejs.cn/en-us/docs/web/api/keyboardevent/getmodifierstate/ (accessed 2026-08-14). https://web.nodejs.cn/en-us/docs/web/api/keyboardevent/getmodifierstate/

24 W3C DOM Level 3 KeyboardEvent key Values (2014 WD) — `"Accel"` virtual modifier definition — https://www.w3.org/TR/2014/WD-DOM-Level-3-Events-key-20140612/ (accessed 2026-08-14). https://www.w3.org/TR/2014/WD-DOM-Level-3-Events-key-20140612/

25 WICG Keyboard Map — getLayoutMap, privacy mitigations ("secure contexts… top-level browsing context"), Permissions-Policy `keyboard-map`, layoutchange event — https://wicg.github.io/keyboard-map/ ; https://github.com/WICG/keyboard-map/issues/38 (accessed 2026-08-14). https://wicg.github.io/keyboard-map/

26 Stack Overflow — "How to create KeyboardEvent with specific keyCode" (keyCode not in KeyboardEventInit; defineProperty workaround) — https://stackoverflow.com/questions/40533292/ (accessed 2026-08-14). https://stackoverflow.com/questions/40533292/

27 w3c/uievents issues #220 and #361 — shipping vs spec ordering of keypress/beforeinput; cancelability interop of composition events; textInput — https://github.com/w3c/uievents/issues/220 ; https://github.com/w3c/uievents/issues/361 (accessed 2026-08-14). https://github.com/w3c/uievents/issues/220

28 Stack Overflow — preventing arrow-key scrolling via keydown preventDefault — https://stackoverflow.com/questions/20794691 (accessed 2026-08-14). https://stackoverflow.com/questions/20794691

29 focus-trap issue #1165 — keydown preventDefault to stop Tab focus navigation (and passive-listener conflict) — https://github.com/focus-trap/focus-trap/issues/1165 (accessed 2026-08-14). https://github.com/focus-trap/focus-trap/issues/1165

30 w3c/uievents issue #396 — can `repeat` be true on keyup? (open) — https://github.com/w3c/uievents/issues/396 (accessed 2026-08-14). https://github.com/w3c/uievents/issues/396

31 MDN — `Event.isTrusted` ("true when the event was generated by a user action… false when… dispatched via dispatchEvent") — https://developer.mozilla.org/en-US/docs/Web/API/Event/isTrusted (accessed 2026-08-14). https://developer.mozilla.org/en-US/docs/Web/API/Event/isTrusted

32 Words by Vernacchia — "Simulating JS Events" (synthetic keyboard events don't insert text / trigger default actions) — https://words.byvernacchia.com/blog/2023/04/simulating-js-events/ (accessed 2026-08-14). https://words.byvernacchia.com/blog/2023/04/simulating-js-events/

33 MDN (mirror) — `Document.activeElement` (returns focused element, `<body>` or null) — https://docs.w3cub.com/dom/document/activeelement.html (accessed 2026-08-14). https://docs.w3cub.com/dom/document/activeelement.html

34 WHATWG HTML Standard — focus fixup rule (quoted via allyjs tutorial) — https://html.spec.whatwg.org/multipage/interaction.html#focus-fixup-rule ; https://kicksky.tistory.com/85 (accessed 2026-08-14). https://html.spec.whatwg.org/multipage/interaction.html#focus-fixup-rule

35 MDN — focus/blur don't bubble; focusin/focusout bubble (reference summary) — https://developer.mozilla.org/en-US/docs/Web/API/Element/focus_event ; https://www.goodsunlc.com/archives/101.html (accessed 2026-08-14). https://developer.mozilla.org/en-US/docs/Web/API/Element/focus_event

36 Stuart Memo — "Handling IME events in JavaScript" (browser divergence around compositionend/keyup; Safari 229) — https://www.stum.de/2016/06/24/handling-ime-events-in-javascript/ (accessed 2026-08-14). https://www.stum.de/2016/06/24/handling-ime-events-in-javascript/

37 microsoft/vscode issue #307646 — Chromium 147 spurious deleteContentBackward before compositionstart; `key: 'Process'` traces — https://github.com/microsoft/vscode/issues/307646 (accessed 2026-08-14). https://github.com/microsoft/vscode/issues/307646

38 chromium docs — "The Life of an Input Event in Desktop Chrome UI", docs/ui/input_event/index.md — https://github.com/chromium/chromium/blob/main/docs/ui/input_event/index.md ; https://chromium.googlesource.com/chromium/src/+/main/docs/ui/input_event/index.md (accessed 2026-08-14). https://github.com/chromium/chromium/blob/main/docs/ui/input_event/index.md

39 chromium — content/browser/renderer_host/render_widget_host_impl.cc (`ForwardKeyboardEventWithCommands`, ~L1737–1850) — https://chromium.googlesource.com/chromium/src/+/main/content/browser/renderer_host/render_widget_host_impl.cc ; https://github.com/chromium/chromium/blob/main/content/browser/renderer_host/render_widget_host_impl.cc (accessed 2026-08-14). https://chromium.googlesource.com/chromium/src/+/main/content/browser/renderer_host/render_widget_host_impl.cc

40 chromium design docs — "How Chromium Displays Web Pages" (Life of a "mouse click" message) — https://www.chromium.org/developers/design-documents/displaying-a-web-page-in-chrome/ (accessed 2026-08-14). https://www.chromium.org/developers/design-documents/displaying-a-web-page-in-chrome/

41 chromium design docs — "OS X keyboard handling" ("Keyboard events should not use synchronous IPC calls"; page first, browser second) — https://www.chromium.org/developers/os-x-keyboard-handling/ (accessed 2026-08-14). https://www.chromium.org/developers/os-x-keyboard-handling/

42 chromium — third_party/blink/renderer/platform/widget/input/input_handler_proxy.cc (`PerformEventAttribution` ~L998–1043; `HandleMouseWheel` ~L1097–1177; `DispatchQueuedInputEvents`/`CoalesceEvents` ~L677–685) — https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/platform/widget/input/input_handler_proxy.cc (accessed 2026-08-14). https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/platform/widget/input/input_handler_proxy.cc

43 chromium — third_party/blink/renderer/core/dom/events/event_target.cc (`SetDefaultAddEventListenerOptions` L453–518; `AddEventListenerInternal` incl. signal path L583–680; `FireEventListeners` L960–1096) — https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/dom/events/event_target.cc (accessed 2026-08-14). https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/dom/events/event_target.cc

44 chromium — third_party/blink/renderer/core/events/keyboard_event.cc and keyboard_event.h (constructor L100–142; getters `key()`/`code()`/`repeat()`) — https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/events/keyboard_event.cc (accessed 2026-08-14). https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/events/keyboard_event.cc

45 chromium — third_party/blink/renderer/core/input/keyboard_event_manager.cc (`KeyboardEventManager::KeyEvent`, L226–417) — https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/input/keyboard_event_manager.cc (accessed 2026-08-14). https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/input/keyboard_event_manager.cc

46 chromium — third_party/blink/renderer/core/input/event_handler.cc (`EventHandler::KeyEvent` ~L2429) — https://github.com/chromium/chromium/blob/main/third_party/blink/renderer/core/input/event_handler.cc (accessed 2026-08-14). https://github.com/chromium/chromium/blob/main/third_party/blink/renderer/core/input/event_handler.cc

47 chromium — third_party/blink/renderer/core/dom/events/event_dispatcher.cc (`Dispatch`, `DispatchEventAtCapturing` incl. `HasCaptureListener`/`SkipEventCapture` skip at ~L294–335, `DispatchEventAtBubbling`, `DispatchEventPostProcess`) — https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/dom/events/event_dispatcher.cc (accessed 2026-08-14). https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/dom/events/event_dispatcher.cc

48 chromium — third_party/blink/renderer/core/dom/events/event_path.cc (`EventPath::CalculatePath`, `CalculateAdjustedTargets`, L107–231) — https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/dom/events/event_path.cc (accessed 2026-08-14). https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/dom/events/event_path.cc

49 chromium — third_party/blink/renderer/core/dom/events/node_event_context.cc (`NodeEventContext::HandleLocalEvents`, L53–63) — https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/dom/events/node_event_context.cc (accessed 2026-08-14). https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/dom/events/node_event_context.cc

50 niek.github.io Chrome features mirror of Blink runtime_enabled_features — `SkipEventCapture`: "Improves performance of event dispatching by skipping the capture phase if there are no capture listeners registered on the page" (enabled by default) — https://niek.github.io/chrome-features/ (accessed 2026-08-14). https://niek.github.io/chrome-features/

51 WebKit — Source/WebCore/dom/EventDispatcher.cpp and Source/WebCore/dom/KeyboardEvent.cpp — https://github.com/WebKit/WebKit/blob/main/Source/WebCore/dom/EventDispatcher.cpp ; https://github.com/WebKit/WebKit/blob/main/Source/WebCore/dom/KeyboardEvent.cpp (accessed 2026-08-14). https://github.com/WebKit/WebKit/blob/main/Source/WebCore/dom/EventDispatcher.cpp

52 chromium — third_party/blink/renderer/core/dom/node.cc (`Node::HandleLocalEvents` ~L3368) — https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/dom/node.cc (accessed 2026-08-14). https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/dom/node.cc

53 chromium — third_party/blink/renderer/core/dom/events/event_listener_map.cc (`Add`/`Remove`/`Find`, crbug.com/1420890 crash key, L112–206) — https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/dom/events/event_listener_map.cc (accessed 2026-08-14). https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/dom/events/event_listener_map.cc

54 chromium — third_party/blink/renderer/core/dom/events/event_target.h (`EventListenerVectorSnapshot` = `HeapVector<Member<RegisteredEventListener>, 1>`; "Do not try to optimize it away", ~L384–393) — https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/dom/events/event_target.h (accessed 2026-08-14). https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/dom/events/event_target.h

55 chromium — third_party/blink/renderer/core/dom/events/registered_event_listener.h / .cc (bitfield options; `ShouldFire`) — https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/dom/events/registered_event_listener.h (accessed 2026-08-14). https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/dom/events/registered_event_listener.h

56 chromium — ui/events/keycodes/dom/keycode_converter.cc (`DomCodeToCodeString` L319+, `DomKeyToKeyString` L429+) — https://github.com/nwjs/chromium.src/blob/main/ui/events/keycodes/dom/keycode_converter.cc (accessed 2026-08-14). https://github.com/nwjs/chromium.src/blob/main/ui/events/keycodes/dom/keycode_converter.cc

57 WebKit bug 149584 — "Implement KeyboardEvent.code from the UI Event spec" (PlatformEventFactoryMac.mm key/code mapping tables) — https://www2.webkit.org/show_bug.cgi?id=149584 (accessed 2026-08-14). https://www2.webkit.org/show_bug.cgi?id=149584

58 mozilla-central — dom/events/KeyboardEvent.cpp (`KeyboardEvent::Key`/`Code` → `GetDOMKeyName`/`GetDOMCodeName` per access; RFP spoofing) — https://searchfox.org/mozilla-central/source/dom/events/KeyboardEvent.cpp (accessed 2026-08-14). https://searchfox.org/mozilla-central/source/dom/events/KeyboardEvent.cpp

59 mozilla-central — dom/events/EventDispatcher.cpp (`EventDispatcher::Dispatch`, EventTargetChain, `PerformanceEventTiming::TryGenerateEventTiming`) — https://hg.mozilla.org/mozilla-central/file/tip/dom/events/EventDispatcher.cpp ; https://searchfox.org/mozilla-central/source/dom/events/EventDispatcher.cpp (accessed 2026-08-14). https://hg.mozilla.org/mozilla-central/file/tip/dom/events/EventDispatcher.cpp

60 chromium — third_party/blink/renderer/bindings/core/v8/js_based_event_listener.cc (`JSBasedEventListener::Invoke`, L69–201) — https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/bindings/core/v8/js_based_event_listener.cc (accessed 2026-08-14). https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/bindings/core/v8/js_based_event_listener.cc

61 chromium — "Design of V8 bindings" (V8BindingDesign.md: same wrapper per world; `ScriptWrappable::main_world_wrapper_`, `DOMWrapperMap`) —

https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/bindings/core/v8/V8BindingDesign.md (accessed 2026-08-14). https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/bindings/core/v8/V8BindingDesign.md

62 chromium — third_party/blink/renderer/bindings/core/v8/js_event_listener.cc (`GetEffectiveFunction` per-invocation `handleEvent` Get; `InvokeInternal`) — https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/bindings/core/v8/js_event_listener.cc (accessed 2026-08-14). https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/bindings/core/v8/js_event_listener.cc

63 chromium — third_party/blink/renderer/bindings/core/v8/v8_script_runner.cc (`V8ScriptRunner::CallFunction` L811+: microtask scope, probes, `function->Call`) — https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/bindings/core/v8/v8_script_runner.cc (accessed 2026-08-14). https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/bindings/core/v8/v8_script_runner.cc

64 Exploit-DB 45444 (stack trace showing FireEventListeners → V8AbstractEventListener::handleEvent → InvokeEventHandler → V8EventListener::CallListenerFunction → V8ScriptRunner::CallFunction → v8::Function::Call → Execution::Call) — https://www.exploit-db.com/exploits/45444 (accessed 2026-08-14). https://www.exploit-db.com/exploits/45444

65 whatwg/dom issue #911 and PR #919 — AbortSignal in addEventListener — https://github.com/whatwg/dom/issues/911 ; https://github.com/whatwg/dom/pull/919 (accessed 2026-08-14). https://github.com/whatwg/dom/issues/911

66 Chrome for Developers — "RenderingNG architecture" (input routing: browser → compositor thread → main thread; scroll fast path) — https://developer.chrome.com/docs/chromium/renderingng-architecture ; https://github.com/GoogleChrome/developer.chrome.com/blob/main/site/en/articles/renderingng-architecture/index.md (accessed 2026-08-14). https://developer.chrome.com/docs/chromium/renderingng-architecture

67 Babylon.js forum thread quoting Chromium issue resolution on input-vs-timer scheduling under main-thread saturation (closed as WAI/intentional) — https://forum.babylonjs.com/t/the-latest-version-of-chrome-edge-browser-the-impact-of-keyboard-input-on-timer-tasks/39390 (accessed 2026-08-14). https://forum.babylonjs.com/t/the-latest-version-of-chrome-edge-browser-the-impact-of-keyboard-input-on-timer-tasks/39390

68 chromium — third_party/blink/renderer/core/timing/event_timing.cc (`IsStandardEventType` includes KeyboardEvent; `UIEventTiming` created in `EventDispatcher::Dispatch`) — https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/timing/event_timing.cc (accessed 2026-08-14). https://chromium.googlesource.com/chromium/src/+/main/third_party/blink/renderer/core/timing/event_timing.cc

69 Wikimedia Diff — "Tracking down slow event handlers with Event Timing" (processingEnd−processingStart; timeStamp == startTime correlation) — https://diff.wikimedia.org/2019/06/19/tracking-down-slow-event-handlers-with-event-timing/

(accessed 2026-08-14). https://diff.wikimedia.org/2019/06/19/tracking-down-slow-event-handlers-with-event-timing/

70 WICG/interventions issue #64 — "Default to passive: true on document level wheel/mousewheel event listeners" — https://github.com/WICG/interventions/issues/64 (accessed 2026-08-14). https://github.com/WICG/interventions/issues/64

71 WebKit bug 188370 — "Events handled by input method invoke default event handler" (EventDispatcher regression analysis) — https://bugs.webkit.org/show_bug.cgi?id=188370 (accessed 2026-08-14). https://bugs.webkit.org/show_bug.cgi?id=188370

72 MeasureThat.net — "AddEventListener vs direct", suite 32515 — https://www.measurethat.net/Benchmarks/Show/32515/1/addeventlistener-vs-direct (accessed 2026-08-14). https://www.measurethat.net/Benchmarks/Show/32515/1/addeventlistener-vs-direct

73 MeasureThat.net (benchmarklab mirror) — "Vanilla JS VS Jquery | Click Event Speed", run result 509037 (Chrome 121, macOS) — https://benchmarklab.azurewebsites.net/Benchmarks/ShowResult/509037 (accessed 2026-08-14). https://benchmarklab.azurewebsites.net/Benchmarks/ShowResult/509037

74 BenchmarkLab (MeasureThat mirror) — "Custom Event vs Callback", suite 20888 — https://benchmarklab.azurewebsites.net/Benchmarks/Show/20888/0/custom-event-vs-callback-with-console-log (accessed 2026-08-14). https://benchmarklab.azurewebsites.net/Benchmarks/Show/20888/0/custom-event-vs-callback-with-console-log

75 jsben.ch — "onclick vs addEventListener" — https://jsben.ch/onclick-vs-addeventlistener-0ngwz (accessed 2026-08-14). https://jsben.ch/onclick-vs-addeventlistener-0ngwz

76 Jason Miller — "Event Listeners: Delegation VS Direct Binding" (2020) — https://jasonformat.com/event-delegation-vs-direct-binding/ (accessed 2026-08-14). https://jasonformat.com/event-delegation-vs-direct-binding/

77 jsben.ch — "Key access speed" (Map/object lookup, not KeyboardEvent) — https://jsben.ch/key-access-speed-pn39c (accessed 2026-08-14). https://jsben.ch/key-access-speed-pn39c

78 Subwaymatch (Medium) — "Comparing keyboard shortcut libraries in React — Mousetrap vs Hotkeys" (2019; features only) — https://subwaymatch.medium.com/comparing-keyboard-shortcut-libraries-in-react-mousetrap-vs-hotkeys-634877b4af9e (accessed 2026-08-14). https://subwaymatch.medium.com/comparing-keyboard-shortcut-libraries-in-react-mousetrap-vs-hotkeys-634877b4af9e

79 MDN — "EventTarget.dispatchEvent()" (synchronous dispatch note) — https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/dispatchEvent (accessed 2026-08-14). https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/dispatchEvent

80 MDN content issue #43519 — misleading async-phrasing in dispatchEvent page — https://github.com/mdn/content/issues/43519 (accessed 2026-08-14). https://github.com/mdn/content/issues/43519

81 Stack Overflow — "How to get microsecond timings in JavaScript since Spectre and Meltdown" (quotes MDN; Firefox 59 rounds to 2 ms) — https://stackoverflow.com/questions/50117537/how-to-get-microsecond-timings-in-javascript-since-spectre-and-meltdown (accessed 2026-08-14).

https://stackoverflow.com/questions/50117537/how-to-get-microsecond-timings-in-javascript-since-spectre-and-meltdown

82 GreatFrontEnd — "Explain event delegation in JavaScript" (2021) — https://www.greatfrontend.com/questions/quiz/explain-event-delegation (accessed 2026-08-14). https://www.greatfrontend.com/questions/quiz/explain-event-delegation

83 Dan Luu — "Keyboard latency" (2017, updated 2022) — https://danluu.com/keyboard-latency/ (accessed 2026-08-14). https://danluu.com/keyboard-latency/

84 Dan Luu — "Computer latency: 1977–2017" (2017) — https://danluu.com/input-lag/ (accessed 2026-08-14). https://danluu.com/input-lag/

85 Pavel Fatin — "Typing with pleasure" (2015) — https://pavelfatin.com/typing-with-pleasure/ (accessed 2026-08-14). https://pavelfatin.com/typing-with-pleasure/

86 WICG — EventListenerOptions explainer (passive listeners; Chrome for Android scroll-blocking stats) — https://github.com/WICG/EventListenerOptions/blob/gh-pages/explainer.md (accessed 2026-08-14). https://github.com/WICG/EventListenerOptions/blob/gh-pages/explainer.md

87 Google Developers — "Making touch scrolling fast by default" / scrolling intervention (Chrome 56, Jan 2017) — https://developers.google.com/web/updates/2017/01/scrolling-intervention (accessed 2026-08-14). https://developers.google.com/web/updates/2017/01/scrolling-intervention

88 Ivan Akulov — "Analysis of passive: true" (Chrome 62 Canary, Firefox 55, 2017) — https://gist.github.com/iamakulov/45803e89db2eb44a7a6be33a80ffcab7 (accessed 2026-08-14). https://gist.github.com/iamakulov/45803e89db2eb44a7a6be33a80ffcab7

89 web.dev — "Interaction to Next Paint (INP)" (source in GoogleChrome/web.dev repo) — https://github.com/GoogleChrome/web.dev/blob/main/src/site/content/en/metrics/inp/index.md ; https://web.dev/articles/inp (accessed 2026-08-14). https://github.com/GoogleChrome/web.dev/blob/main/src/site/content/en/metrics/inp/index.md

90 This Dot — "New Core Web Vitals and How They Work" (2024; keystroke = keydown/keypress/keyup, max duration) — https://www.thisdot.co/blog/new-core-web-vitals-and-how-they-work (accessed 2026-08-14). https://www.thisdot.co/blog/new-core-web-vitals-and-how-they-work

91 WICG scheduling-apis — "yield-and-continuation" explainer (Chrome Android trace analysis of slow interactions) — https://github.com/WICG/scheduling-apis/blob/main/explainers/yield-and-continuation.md (accessed 2026-08-14). https://github.com/WICG/scheduling-apis/blob/main/explainers/yield-and-continuation.md

92 SpeedCurve — "First Input Delay" RUM data (2018; median 3 ms, p95 30 ms) — https://www.speedcurve.com/blog/first-input-delay/ (accessed 2026-08-14). https://www.speedcurve.com/blog/first-input-delay/

93 pagespeed-optimierung.de — "Core Web Vitals 2026" citing HTTP Archive 2025 (65% good INP vs 93% FID; secondary) — https://www.pagespeed-optimierung.de/en/blog/core-web-vitals-2026/ (accessed 2026-08-14). https://www.pagespeed-optimierung.de/en/blog/core-web-vitals-2026/

94 DebugBear — "Getting Started With scheduler.yield" (Chrome 129+ support noted) — https://www.debugbear.com/blog/scheduler-yield (accessed 2026-08-14).

https://www.debugbear.com/blog/scheduler-yield

95 Perf Planet Calendar 2024 — "Breaking Up with Long Tasks…" (scheduler.yield ~1 s vs setTimeout >2 min) — https://calendar.perfplanet.com/2024/breaking-up-with-long-tasks-or-how-i-learned-to-group-loops-and-wield-the-yield/ (accessed 2026-08-14). https://calendar.perfplanet.com/2024/breaking-up-with-long-tasks-or-how-i-learned-to-group-loops-and-wield-the-yield/

96 VS Code source — `src/vs/platform/keybinding/common/abstractKeybindingService.ts` — https://raw.githubusercontent.com/microsoft/vscode/main/src/vs/platform/keybinding/common/abstractKeybindingService.ts (accessed 2026-08-14). https://raw.githubusercontent.com/microsoft/vscode/main/src/vs/platform/keybinding/common/abstractKeybindingService.ts

97 VS Code source — `src/vs/base/browser/keyboardEvent.ts` (`StandardKeyboardEvent`, `extractKeyCode`) — https://raw.githubusercontent.com/microsoft/vscode/main/src/vs/base/browser/keyboardEvent.ts (accessed 2026-08-14). https://raw.githubusercontent.com/microsoft/vscode/main/src/vs/base/browser/keyboardEvent.ts

98 VS Code wiki — Keybinding Issues (scan-code dispatch, AltGraph policy, Linux layout cache) — https://github.com/microsoft/vscode/wiki/Keybinding-Issues (accessed 2026-08-14). https://github.com/microsoft/vscode/wiki/Keybinding-Issues

99 VS Code source — `src/vs/platform/keybinding/common/keybindingResolver.ts` — https://raw.githubusercontent.com/microsoft/vscode/main/src/vs/platform/keybinding/common/keybindingResolver.ts (accessed 2026-08-14). https://raw.githubusercontent.com/microsoft/vscode/main/src/vs/platform/keybinding/common/keybindingResolver.ts

100 CodeMirror 6 source — `packages/view/src/keymap.ts` — https://raw.githubusercontent.com/codemirror/view/main/src/keymap.ts (accessed 2026-08-14). https://raw.githubusercontent.com/codemirror/view/main/src/keymap.ts

101 CodeMirror reference manual — `@codemirror/view` keymap facet, KeyBinding flags — https://codemirror.net/docs/ref/ (accessed 2026-08-14). https://codemirror.net/docs/ref/

102 ProseMirror source — `prosemirror-keymap/src/keymap.ts` — https://raw.githubusercontent.com/ProseMirror/prosemirror-keymap/master/src/keymap.ts (accessed 2026-08-14). https://raw.githubusercontent.com/ProseMirror/prosemirror-keymap/master/src/keymap.ts

103 xterm.js source — `src/common/input/Keyboard.ts` (`evaluateKeyboardEvent`) — https://raw.githubusercontent.com/xtermjs/xterm.js/master/src/common/input/Keyboard.ts (accessed 2026-08-14). https://raw.githubusercontent.com/xtermjs/xterm.js/master/src/common/input/Keyboard.ts

104 hotkeys-js README (options, scopes, filter, getAllKeyCodes shape) — https://github.com/jaywcjlove/hotkeys-js (accessed 2026-08-14). https://github.com/jaywcjlove/hotkeys-js

105 hotkeys-js source — `src/index.ts` — https://raw.githubusercontent.com/jaywcjlove/hotkeys-js/master/src/index.ts (accessed 2026-08-14). https://raw.githubusercontent.com/jaywcjlove/hotkeys-js/master/src/index.ts

106 hotkeys-js source — `src/utils.ts` (`getLayoutIndependentKeyCode`, `compareArray`) — https://raw.githubusercontent.com/jaywcjlove/hotkeys-js/master/src/utils.ts (accessed 2026-08-14). https://raw.githubusercontent.com/jaywcjlove/hotkeys-js/master/src/utils.ts

107 hotkeys-js source — `src/var.ts` (`_keyMap`, `_modifier`, `modifierMap`, `_handlers`) — https://raw.githubusercontent.com/jaywcjlove/hotkeys-js/master/src/var.ts (accessed 2026-08-14). https://raw.githubusercontent.com/jaywcjlove/hotkeys-js/master/src/var.ts

108 hotkeys-js `dispatch`/`eventHandler` annotated source mirror — https://www.cnblogs.com/mq0036/p/4955896.html (accessed 2026-08-14). https://www.cnblogs.com/mq0036/p/4955896.html

109 Mousetrap source — `mousetrap.js` v1.6.5 — https://raw.githubusercontent.com/ccampbell/mousetrap/master/mousetrap.js (accessed 2026-08-14). https://raw.githubusercontent.com/ccampbell/mousetrap/master/mousetrap.js

110 tinykeys source — `src/tinykeys.ts` — https://raw.githubusercontent.com/jamiebuilds/tinykeys/main/src/tinykeys.ts (accessed 2026-08-14). https://raw.githubusercontent.com/jamiebuilds/tinykeys/main/src/tinykeys.ts

111 kbar source — `src/InternalEvents.tsx` (tinykeys usage, WeakSet wrap, timeout 400) — https://raw.githubusercontent.com/timc1/kbar/main/src/InternalEvents.tsx (accessed 2026-08-14). https://raw.githubusercontent.com/timc1/kbar/main/src/InternalEvents.tsx

112 kbar `src` listing (vendored `tinykeys.ts`, deps: fast-equals, fuse.js, @tanstack/react-virtual) — https://api.github.com/repos/timc1/kbar/contents/src ; https://raw.githubusercontent.com/timc1/kbar/main/package.json (accessed 2026-08-14). https://api.github.com/repos/timc1/kbar/contents/src

113 react-hotkeys-hook `useHotkeys.ts` source (via issue #989 gist mirror) — https://gist.github.com/The-Podsiadly/988d68762d6f0999877e249c0ade3b76 (accessed 2026-08-14). https://gist.github.com/The-Podsiadly/988d68762d6f0999877e249c0ade3b76

114 react-hotkeys-hook v5.0.0 release notes (code-based matching, `useKey`) — https://github.com/JohannesKlauss/react-hotkeys-hook/releases (accessed 2026-08-14). https://github.com/JohannesKlauss/react-hotkeys-hook/releases

115 React v17.0 blog post — Changes to Event Delegation — https://legacy.reactjs.org/blog/2020/10/20/react-v17.html (accessed 2026-08-14). https://legacy.reactjs.org/blog/2020/10/20/react-v17.html

116 React releases/changelog (PR #18195, #19659, #19221, #18287, #18969) — https://github.com/facebook/react/releases?after=v16.13.1 (accessed 2026-08-14). https://github.com/facebook/react/releases?after=v16.13.1

117 React `getEventKey` normalization (legacy key values, keyCode→key polyfill) — analysis — https://blog.huli.tw/2019/03/24/en/react-keypress-keydown/ (accessed 2026-08-14). https://blog.huli.tw/2019/03/24/en/react-keypress-keydown/

118 React v17 RC blog post — event delegation rationale, capture-phase guidance — https://legacy.reactjs.org/blog/2020/08/10/react-v17-rc.html (accessed 2026-08-14). https://legacy.reactjs.org/blog/2020/08/10/react-v17-rc.html

119 VS Code issue #129625 — keybinding when-clause resolving slows workbench startup — https://github.com/microsoft/vscode/issues/129625 (accessed 2026-08-14). https://github.com/microsoft/vscode/issues/129625

120 Stack Overflow — "How to read jsben.ch benchmark result" (harness internals: fixed-time window, performance.now loop) — https://stackoverflow.com/questions/69189911/how-to-read-jsben-ch-benchmark-result (accessed 2026-08-14).
https://stackoverflow.com/questions/69189911/how-to-read-jsben-ch-benchmark-result